Finite-Dimensional Type I von Neumann Algebras in PyTorch: A GPU-Accelerated Framework for Random Block-Diagonal Operators

Irina Nikolaeva Andrej Novikov

Abstract

`torch_vn_algebra` is a PyTorch library for batched, differentiable computations on CPUs and GPUs in the finite-dimensional Type I von Neumann algebras $\bigoplus_c M_{k_c}(\mathbb{C})$ of quantum systems with conserved charges or superselection rules: random operators with prescribed spectra, states, channels within and between sectors, spin chains, Lindblad dynamics, inclusions of algebras, symmetry reduction and constrained optimisation. This paper documents how it is built. Operators are lazy recipes that carry their mathematical properties, so the right kernel is chosen without checks and nothing is computed that is not read; extreme eigenvalues, traces, inverses and, on request, spectral decompositions are cached. The speed comes from a padded block layout with one batched call per block size, a single batched Hermitian eigensolver behind spectra, functions, singular values and polar decompositions, direct formulas instead of products, no hidden host synchronisation, sparse sector Hamiltonians with batched Krylov solvers, and sparse jumps between sectors for the Lindblad solvers; each of these decisions is measured against the naive alternative. The library is validated against exact results (Haar and Weingarten moments, Page's formula, circular ensembles). As its main application we study symmetry verification for a noisy XY mixer of the quantum alternating operator ansatz, whose Hamming weight sectors form the algebra: with all noise models and rates in one batch, we show that post-selection on the weight removes amplitude damping exactly at zero temperature, suppresses thermal relaxation from first to second order, and leaves dephasing undetected, with an infidelity given to first order by one integral over the noiseless trajectory. Four further applications, with their own companion papers, are summarised.

Program summary

Program title: `torch_vn_algebra`

Developer's repository link: `https://github.com/andre-a-no/von_neumann_type_i`

Licensing provisions: MIT

Programming language: Python (≥ 3.8), PyTorch (≥ 1.13), NumPy (≥ 1.21)

Nature of problem: Numerical experiments in finite-dimensional Type I von Neumann algebras $\bigoplus_c M_{k_c}(\mathbb{C})$, which describe quantum systems with superselection rules or conserved charges: random operators with prescribed spectra, several trace functionals, states, channels within and between sectors, symmetric tensor products, spin chains, Hamiltonian and Lindblad dynamics with jumps between sectors (noisy circuits with a conserved charge), extremal problems under spectral constraints, inclusions of algebras, symmetry reduction, and quantum Rényi divergences.

Solution method: Batched block-diagonal tensors processed by PyTorch linear algebra on CPUs and GPUs, one call per block size; lazy operators that carry their properties, with caches for

spectra; exact Haar sampling; spectral functional calculus; Kraus representations; Lanczos and Krylov methods for sparse sector Hamiltonians; sparse jumps between sectors for the Lindblad solvers; differentiable parametrisations with batched multistart gradient optimisation; run-time and memory estimates before expensive steps; inclusion matrices; randomised Wedderburn decomposition from random elements of the commutant; highest-weight (Yamanouchi) bases and exact Clebsch–Gordan coefficients.

Additional comments: more than 800 unit tests, run in continuous integration; every number, table and figure of this paper is produced by the scripts and examples in the repository and regenerated by `scripts/run_all.sh`.

1 Introduction

Where block-diagonal algebras arise. Whenever a conserved quantity, a symmetry or a superselection rule splits a Hilbert space into sectors, the physically relevant operators are block diagonal. Their algebra is a finite-dimensional Type I von Neumann algebra

$$\mathcal{M} = \bigoplus_{c=1}^{C} M_{k_c}(\mathbb{C}) \quad \text{acting on} \quad \mathcal{H} = \bigoplus_{c=1}^{C} \mathbb{C}^{k_c}, \tag{1}$$

where C is the number of sectors (direct summands) and $M_k(\mathbb{C})$ is the algebra of complex $k \times k$ matrices. Every finite-dimensional von Neumann algebra is $*$-isomorphic to one of this form [25], and its centre is spanned by the sector projections. The library also accepts nominal factor sizes $n_c \geq k_c$: this describes computations in the corner $p\mathcal{M}_0 p \cong \bigoplus_c M_{k_c}(\mathbb{C})$ of a larger algebra $\mathcal{M}_0 = \bigoplus_c M_{n_c}(\mathbb{C})$, with $p = \bigoplus_c p_c$ and p_c a projection of rank k_c. The corner is again of the form (1); all operators, traces and normalisations refer to it, and in most applications $k_c = n_c$.

From a symmetry to the algebra. The link between physics and (1) is the commutant. If a group G (or a set of conserved quantities) acts unitarily on $\mathcal{H}$ and $\mathcal{H} \cong \bigoplus_\lambda V_\lambda \otimes \mathbb{C}^{m_\lambda}$ is its decomposition into irreducible representations V_λ with multiplicities m_λ, then the operators commuting with the action form the algebra $\bigoplus_\lambda \mathbb{1}_{V_\lambda} \otimes M_{m_\lambda}(\mathbb{C}) \cong \bigoplus_\lambda M_{m_\lambda}(\mathbb{C})$. For one conserved self-adjoint charge Q (particle number, magnetisation, Hamming weight) the irreducibles are one-dimensional and m_λ is the dimension of the eigenspace of Q with eigenvalue λ. A Hamiltonian that conserves Q, a time evolution generated by it, and every observable compatible with a superselection rule for Q lie in this commutant; the sectors are the eigenvalues of Q, and the centre consists of the functions of Q. Physical modelling with a symmetry is therefore computation in an algebra of the form (1); for non-abelian symmetries the block sizes m_λ are smaller than the dimensions of the charge sectors, and Section 8 computes the decomposition numerically from the group or its generators.

Examples are abundant:

- superselection rules for charge, parity or particle number [26, 27] and the classical-quantum structure produced by decoherence and einselection [28];
- many-body Hamiltonians with conserved charges, whose exact diagonalisation and dynamics are carried out sector by sector [29, 30], and symmetry-resolved entanglement [31, 32];
- quantum information: decoherence-free subspaces and noiseless subsystems [33, 34], operator-algebra quantum error correction [35], and quantum algorithms that preserve a conserved quantity, such as the XY mixers of the quantum alternating operator ansatz, which keep the Hamming weight [38];

- random matrix theory, where symmetries decompose ensembles into blocks and spectral statistics are meaningful only inside a block [39, 15];
- machine learning: equivariant layers are the linear maps that commute with a group representation [40, 41]; by Schur's lemma [81], for a complex unitary representation on one space they form an algebra of the form (1), whose block sizes are the multiplicities of the irreducible representations, and equivariant layers between two representations are the corresponding off-diagonal blocks (for real representations the blocks can also be real or quaternionic [81, §3.5]); random-matrix methods are used to analyse trained weights [42];
- operator theory, where inequalities between traces of products characterise the trace and the centre of an operator algebra [5, 6, 7, 8], and the size of their failure was studied numerically against the Michelson contrast [9].

On $\mathcal{M}$ the trace is not unique: every positive weighting of the sectors gives one, and statements such as "trace preserving" or "the dual map" depend on that choice as soon as maps change the sector structure.

What existing software offers. Table 1 compares the features relevant here. Dense and sparse linear algebra (NumPy/SciPy [43, 44], with ARPACK [45] for sparse eigenproblems) handles single matrices; QuTiP [10, 11] and the quantum-information module of Qiskit [12] represent states, operators and channels on one Hilbert space, and PIQS, part of QuTiP, exploits permutational symmetry for the Lindblad dynamics of N two-level systems [98]; `dynamiqs` [13] adds batching, GPUs and automatic differentiation for Schrödinger and Lindblad dynamics; QuSpin [30] builds symmetry-reduced bases for exact diagonalisation; tensor-network libraries (ITensor [46], TeNPy [48]) store charge-conserving tensors in block-sparse form, and YASTN [47], the closest to this work in its software stack, does so on a PyTorch backend; RTNI [49] integrates Haar-random tensor networks symbolically. None of them treats the algebra (1) itself, with its centre, several traces, states and maps between algebras of different sector structure, as the basic object, and none combines it with batched sampling and optimisation over operators with prescribed spectra.

	NumPy/ SciPy	QuTiP	Qiskit q. info	`dynamiqs`	QuSpin	ITensor/ TeNPy	this work
GPU execution	–	(✓)	–	✓	–	(✓)	✓
batched over many operators	(✓)	–	–	✓	–	–	✓
automatic differentiation	–	(✓)	–	✓	–	(✓)	✓
symmetry sectors of many-body systems	–	–	–	–	✓	✓	✓
algebra $\bigoplus_c M_{k_c}$ with centre and several traces	–	–	–	–	–	–	✓
channels within and between sectors	–	(✓)	(✓)	–	–	–	✓
prescribed spectra, Haar/COE/CSE per sector	(✓)	(✓)	(✓)	–	–	–	✓
optimisation under spectral constraints	–	–	–	–	–	–	✓
large single systems	✓	✓	✓	✓	✓	✓	(✓)

Table 1: Features relevant to computations in the algebra (1), to the best of our knowledge at the time of writing (✓: supported; (✓): partly, through plug-ins, or with user code; –: not supported). The last row is where the specialised packages are the better choice. The QuTiP column refers to QuTiP 5 [11]. YASTN [47] has the marks of ITensor/TeNPy, with GPU execution and automatic differentiation through its PyTorch backend.

Why not one large matrix? Nothing forces one to treat $\mathcal{M}$ as a separate object: it embeds into $M_D(\mathbb{C})$, $D = \sum_c k_c$, as the block-diagonal matrices, and every package above handles a single $D \times D$

matrix (the case $C = 1$ of (1)). The embedding is exact for linear algebra, so the question is only what it costs and what has to be supplied by hand.

- *Cost.* A product, an inverse or a spectral decomposition in M_D costs $O(D^3)$ time and $O(D^2)$ memory, in $\mathcal{M}$ it costs $\sum_c k_c^3$ and $\sum_c k_c^2$. For C equal blocks this is a factor C^2 in time and C in memory; for the half-filling sector of a chain of $L = 16$ spins ($D = 2^{16}$, largest block 12870) the dense M_D computation is out of reach, the block computation is routine.
- *Measures.* The unitary group of $\mathcal{M}$ is $\prod_c U(k_c)$, for $C \geq 2$ a subgroup of $U(D)$ of measure zero. A Haar-random element of $U(D)$ is not in $\mathcal{M}$, and a random element of $\mathcal{M}$ has to be produced block by block. The same holds for random states, for the circular ensembles and for any optimisation: an unconstrained gradient step in M_D leaves $\mathcal{M}$.
- *Traces.* On M_D the trace is unique up to a factor; on $\mathcal{M}$ the faithful traces form a C-parameter family $\sum_c w_c \operatorname{Tr}_c$. Densities, "trace preserving" and the dual of a map between algebras with different sectors depend on the choice of the weights; on the embedded matrices this information is no longer visible.
- *Centre.* The centre of $\mathcal{M}$ (the sector projections) is what physicists call the classical or superselected part of the system; conditioning on a sector, post-selection and sector-resolved quantities are operations with the centre, and they have to be reconstructed from the block indices when $\mathcal{M}$ is stored as one matrix.

The library keeps these four pieces of structure inside the data type. For $C = 1$ it reduces to batched linear algebra in $M_k(\mathbb{C})$, and all its functions accept that case. The first version of this library [4] shows that the bookkeeping is error-prone: its complex Haar sampler was not Haar distributed, its circular ensembles were not the circular ensembles, and several conclusions drawn from Monte Carlo samples were wrong; all of this is corrected and tested here.

Challenges. Turning the algebra into an efficient and reliable tool raises concrete problems, each addressed by a design decision described in Section 3: (i) heterogeneous block sizes have to be mapped onto batched GPU kernels; (ii) trace conventions and dual maps must stay consistent when maps change the sectors; (iii) random ensembles must be exactly Haar (or COE/CSE) in every block; (iv) gradients propagated through eigen- or singular-vector decompositions (as in $f(A)$ or $|X|$) are numerically singular at degenerate spectra, exactly where extremal operators tend to lie; (v) extremal values sit on sets of tiny measure that random sampling does not reach in higher dimensions (Section 6); (vi) batched workloads easily exceed the time or memory budget of a device; (vii) the number field (real or complex) and the precision change both cost and results (Section 3).

Objective and scope. `torch_vn_algebra` is a library for numerical experiments *in* the algebra (1): sampling and optimising operators, states and channels, evaluating trace and spectral functionals, and formulating physical models (spin chains, open systems, noisy symmetric quantum circuits) whose symmetry is encoded in the sector structure. It is not a circuit simulator and not a tensor-network code, and for single large systems the specialised packages above remain the better choice; Section 11 quantifies where the library gains and where it does not.

Contributions and outline. The paper is a technical description of the library: what it computes, how the computation is organised, and what each design decision costs or saves. Section 2 fixes notation and conventions. Section 3 describes the architecture: the software stack and workflow, the data layout, lazy evaluation with property flags and caches (Section 3), the sources of speed with measurements (Section 3), the functional calculus and its derivatives, GPUs, the number field and

precision, and the safeguards for time and memory. Section 4 gives short complete programs, Section 5 validates the library against exact results, Section 6 describes constrained optimisation, which replaces Monte Carlo sampling where extremes matter, and rediscovers the data-processing thresholds of the Rényi divergences, Section 7 charge-conserving tensor products, spin chains, Krylov solvers and jumps between sectors, and Section 8 inclusions of algebras, the Wedderburn decomposition of a symmetry and SU(2) multiplets. The main application, Section 9, studies symmetry verification for a noisy XY mixer, where the Hamming-weight sectors form the algebra; Section 10 summarises four applications reported in companion papers. Section 11 compares with NumPy/SciPy, Section 12 reports performance, and Section 13 discusses limitations.

2 Mathematical setting

Algebra and traces. Let $\mathcal{M} = \bigoplus_{c=1}^{C} M_{k_c}(\mathbb{C})$ act on $\mathcal{H} = \bigoplus_c \mathbb{C}^{k_c}$; $A \in \mathcal{M}$ is block diagonal, $A = \bigoplus_c A_c$. Every trace on $\mathcal{M}$ is $\mathrm{tr}_w(A) = \sum_c w_c \operatorname{Tr} A_c$ with weights $w_c \geq 0$, and it is faithful iff all $w_c > 0$. The library implements

$$\mathrm{Tr}_{\mathrm{blunt}}: \ w_c = 1, \qquad \mathrm{Tr}_{\mathrm{norm}}: \ w_c = \tfrac{1}{k_c}, \qquad \tau_{\mathrm{vN}}: \ w_c = \tfrac{1}{Ck_c},$$

where τ_{vN} is the faithful normal tracial state that gives every sector the same weight (the library allows empty sectors, $k_c = 0$; they carry no weight, and C in τ_{vN} counts the non-empty ones). For $A \in \mathcal{M}$, $|A| = (A^*A)^{1/2}$; for $A \geq 0$ the Michelson contrast is $\Delta(A) = (\lambda_{\max} - \lambda_{\min})/(\lambda_{\max} + \lambda_{\min}) \in [0, 1]$, with the extreme eigenvalues taken over all sectors, for $A \neq 0$ (the library returns $\Delta(0) = 0$); for $A \geq 0$, $A \neq 0$, $\Delta(A) = 0$ if and only if A is a multiple of the identity.

States. A state ω on $\mathcal{M}$ is represented by its density $\rho \geq 0$ with respect to $\mathrm{Tr}_{\mathrm{blunt}}$, $\omega(A) = \mathrm{Tr}(\rho A)$, $\sum_c \operatorname{Tr} \rho_c = 1$. The restriction of ω to the centre is the classical distribution $p_c = \operatorname{Tr} \rho_c$ over sectors. The density of the same state with respect to tr_w is $\rho_c^{(w)} = \rho_c / w_c$; the library converts between the conventions explicitly, because statements such as "trace preserving" or "the dual map" depend on them as soon as the sectors change.

Maps between sectors. Let $\mathcal{M}_{\mathrm{in}} = \bigoplus_c M_{k_c}$ and $\mathcal{M}_{\mathrm{out}} = \bigoplus_d M_{m_d}$. Every normal completely positive map $\Phi : \mathcal{M}_{\mathrm{in}} \to \mathcal{M}_{\mathrm{out}}$ (in the Schrödinger picture) is a sum of completely positive maps between blocks,

$$\Phi(\rho)_d = \sum_c \sum_i K_i^{dc} \, \rho_c \, (K_i^{dc})^*, \qquad K_i^{dc} : \mathbb{C}^{k_c} \to \mathbb{C}^{m_d}.$$

It preserves $\mathrm{Tr}_{\mathrm{blunt}}$ iff $\sum_{d,i} (K_i^{dc})^* K_i^{dc} = 1_c$ for all c, and $T_{dc} = \operatorname{Tr} \Phi_{dc}(\rho_c) / \operatorname{Tr} \rho_c$ are the transition probabilities between sectors for the input state ρ (defined for $\operatorname{Tr} \rho_c > 0$) (independent of ρ iff every $\sum_i (K_i^{dc})^* K_i^{dc}$ is a multiple of 1_c; by default the library uses the maximally mixed state of each input sector). The dual with respect to tr_w on both sides is $\Phi^{*,w}(A)_c = \sum_{d,i} (w_d^{\mathrm{out}} / w_c^{\mathrm{in}}) (K_i^{dc})^* A_d K_i^{dc}$; for sector-preserving maps on one algebra the weights cancel and all three traces give the same dual.

Tensor products and fusion of sectors. $\left(\bigoplus_c M_{k_c}\right) \otimes \left(\bigoplus_d M_{m_d}\right) = \bigoplus_{(c,d)} M_{k_c m_d}$ is again of Type I with sectors labelled by pairs. If the sectors carry conserved charges q_c, q'_d (particle numbers, magnetisation), the operators of the composite system that conserve only the *total* charge form the larger algebra

$$\mathcal{M}_1 \hat{\otimes} \mathcal{M}_2 = \bigoplus_q M_{n_q}, \qquad n_q = \sum_{q_c + q'_d = q} k_c m_d,$$

in which all pairs with the same total charge are merged into one sector. It contains $\mathcal{M}_1 \otimes \mathcal{M}_2$ (block-diagonally in every merged sector), but also operators such as $S_i^+ S_j^-$ that move charge between the factors. For additive charges, and distinct charges on distinct sectors of $\mathcal{M}_1$, the partial trace $(\mathrm{Tr}_2 X)_c = \sum_d \mathrm{Tr}_{m_d} X_{(c,d),(c,d)}$ still lands in $\mathcal{M}_1$: a block between pairs $(c,d) \neq (c',d)$ with equal total charge would need $q_c = q_{c'}$, hence $c = c'$. In both cases the partial trace is a map between sectors in the sense above, and its dual with respect to $\mathrm{Tr}_{\mathrm{blunt}}$ is the embedding $A \mapsto A \otimes 1$.

Spin chains. For L spins 1/2 with conserved S^z_{tot}, iterating the fused product of single-site algebras $M_1 \oplus M_1$ (charges $0, 1$) gives $\mathcal{M} = \bigoplus_{N=0}^{L} M_{\binom{L}{N}}(\mathbb{C})$, N the number of up spins, which contains every S^z_{tot}-conserving Hamiltonian, e.g. the XXZ chain $H = \sum_{\langle ij \rangle} \left[\frac{J}{2}(S_i^+ S_j^- + S_i^- S_j^+) + J\Delta S_i^z S_j^z \right] + \sum_i h_i S_i^z$ (the anisotropy Δ is unrelated to the Michelson contrast $\Delta(\cdot)$).

Dynamics. For $H = H^* \in \mathcal{M}$ and jump operators $L_j \in \mathcal{M}$ the Gorini–Kossakowski–Sudarshan–Lindblad equation $\dot{\rho} = -i[H, \rho] + \sum_j \gamma_j (L_j \rho L_j^* - \frac{1}{2}\{L_j^* L_j, \rho\})$ preserves every sector, so the sector probabilities p_c are conserved (superselection). Jumps between sectors, such as particle loss, are described by maps between sectors as above (Section 7).

3 Design and implementation

Software stack. The library is pure Python on top of PyTorch [87] ($\geq$ 1.13) and NumPy [43]. All dense linear algebra goes through `torch.linalg`, i.e. LAPACK (MKL or OpenBLAS) on CPUs and cuSOLVER, cuBLAS and MAGMA on NVIDIA GPUs; sparse products use PyTorch's CSR tensors. Automatic differentiation is PyTorch's reverse mode. The validation and paper scripts additionally use SciPy, pandas, Matplotlib and mpmath (for the high-precision certification of Section 6.1). Packaging follows `pyproject.toml`; the test suite runs in continuous integration on Python 3.8–3.13 (CPU) and with the oldest supported versions (Python 3.8, PyTorch 1.13, NumPy 1.21), and the paper scripts state the device and mode in the table captions and record the matmul precision, the PyTorch, CUDA and cuDNN versions and the compute capability in a JSON file next to every table.

Workflow. A computation proceeds in four steps. (1) An algebra is declared by its sector sizes, optionally sector charges, number field, precision and device (`TypeIAlgebra`, or a model such as `SpinChain` that creates it). (2) Objects are created in it: random operators with prescribed spectra, states, channels, Hamiltonians, or differentiable parametrisations. (3) They are combined lazily (products, functional calculus, channel application, tensor products, time evolution). (4) Functionals are evaluated for the whole batch at once (traces, spectra, contrasts, entropies, sector probabilities) and reduced to statistics, or fed into an optimiser that updates the parameters of step (2). Every step keeps the batch dimension, so the same code serves one operator and 10^5 samples.

Data layout. An operator is a tensor of shape $(B, C, k_{\max}, k_{\max})$: B samples, C sectors, block c in the top-left $k_c \times k_c$ corner and zeros elsewhere. Linear algebra is applied to the batch of active blocks of each sector, so one call to a batched LAPACK/cuSOLVER routine serves all samples. The padding costs memory when the sector sizes differ strongly (for a chain of 14 spins, 77 % of a padded operator is zeros); the library warns in that case, and `SpinChain` lets one work in single sectors $M_{\binom{L}{N}}$ without padding, which is what dense diagonalisation needs anyway.

challenge	solution in the library
(i) blocks of different size on batched kernels	padded $(B, C, k_{\max}, k_{\max})$ layout; products and traces in one batched call, decompositions in one batched call per distinct block size; single-sector algebras and sparse sector Hamiltonians where padding would dominate
(ii) conventions between sectors	states stored as $\mathrm{Tr}_{\mathrm{blunt}}$-densities with explicit conversions; duals of maps between sectors computed with the weights of the chosen trace
(iii) exact random ensembles	Mezzadri's QR algorithm with phase correction, $\det^{-1/n}$ for $SU(n)$, $W^T W$ and $W^R W$ for COE/CSE, validated against Weingarten moments and spacing statistics
(iv) singular gradients at degenerate spectra	functional calculus differentiated by the Daleckii–Krein formula (divided differences, finite at degenerate spectra); parametrisations that build operators from their spectra in the optimisation, exact $\lvert X\rvert$ from the parametrisation
(v) extremes missed by sampling	batched multistart optimisation with exact contrast constraints
(vi) time and memory budgets	probing of batched kernels with an ETA warning, memory estimates checked against the free device memory, ETA for time stepping
(vii) field and precision	two switches per algebra, conversions between them, measured trade-offs

Table 2: Implementation challenges (Section 1) and how they are addressed.

Objects. `Operator` (with `DensityMatrix` as a subclass for states), `Channel` (sector preserving) and `InterSectorChannel` (between sectors or algebras) are thin wrappers around such tensors, and `SectorJump` is a sparse jump operator for the Lindblad solvers (below); Table 3 lists the modules.

Lazy evaluation and caches

Operators are recipes until used. An `Operator` holds either its matrix or a *generator*, a closure that computes it. Sums, products, adjoints, scalar multiples, functions $f(A)$, channel images $\Phi(\rho)$ and maps between sectors return new operators that hold only a generator; nothing is computed. The matrix is formed on the first access to `.matrix`: the generator runs, its result is stored, and the generator is dropped, because it holds references to the parent operators and would otherwise keep their matrices alive. A long expression therefore costs exactly the products that are needed for the requested result, intermediates can be freed as soon as the result exists, and branches that are never read are never computed. The batch size of a lazy operator is propagated without computing it, so shapes and memory estimates are available before any work is done. Laziness defers and skips work; it does not fuse kernels. Where fusion matters, the library uses direct formulas instead of composing operations: $\mathrm{Tr}(\rho A) = \sum_{ij} \rho_{ij} A_{ji}$ in $O(k^2)$ per block instead of forming ρA, the sector probabilities of a channel image as one contraction $\sum K_{mi} \rho_{ij} \overline{K_{mj}}$ without forming the image.

Properties travel with the recipe. Every operator carries flags (self-adjoint, normal, positive, invertible, projection) that are known, unknown, or checked. Operations set the flags that hold mathematically: the sum of self-adjoint operators is self-adjoint, of positive ones positive; a product of invertible operators is invertible; the adjoint keeps all flags; an operator built from its spectrum is self-adjoint (positive, if the spectrum is). Flags select the kernel: a self-adjoint operator uses the Hermitian eigensolver for its norm, trace norm, inverse and $|A|$, never an SVD, and `lambda_max` of an operator built from its spectrum is read from the spectrum without any decomposition. A flag that is unknown is computed once, on demand, and stored. Flags passed by the user are trusted on eager operators and validated when a lazy operator is materialised, with all checks reduced to one host synchronisation; the tolerances are kept as device tensors.

module	content
`algebra`	`TypeIAlgebra`, `Operator`: random operators, unitary ensembles, traces, functional calculus
`states`	`DensityMatrix`, random states, Gibbs states, Lüders rule, entropies, fidelity
`channels`	`Channel`, `InterSectorChannel`, Choi matrices, standard and random channels
`composite`	plain and fused (charge-conserving) tensor products, `kron`, partial traces
`chains`	`SpinChain`: $U(1)$ and momentum sectors, XXZ Hamiltonians with batched disorder, sparse form, reduced states, jump operators between sectors
`krylov`	sparse sector Hamiltonians, batched Lanczos ground states, Krylov $e^{-itH}\psi$
`dynamics`	exact Schrödinger/von Neumann evolution, RK4, Lindblad equation, $e^{t\mathcal{L}}$
`functional`	functional calculus: self-adjoint, normal, singular values, holomorphic (contour); spectral cache
`optimize`	parametrisations with prescribed contrast, batched multistart `extremize`
`cost`	run-time and memory estimates, warnings and errors
`hilbert_space`	the padded block layout of $\bigoplus_c \mathbb{C}^{k_c}$ and its random subspace bases
`inclusions`	`Inclusion`: inclusions $\mathcal{N} \subset \mathcal{M}$ by inclusion matrices, conditional expectations, Markov trace, Jones, Watatani and Pimsner–Popa indices, basic construction, spin-chain tower
`decompose`	numerical Wedderburn decomposition of the algebra generated by matrices or a finite group; commutants; real, complex and quaternionic types
`su2`	`SU2Chain`: SU(2) multiplets of spin-$\frac{1}{2}$ chains, highest-weight bases, Clebsch–Gordan coefficients, Wigner–Eckart theorem
`divergences`	Petz and sandwiched Rényi divergences and their limits, Petz and universal recovery maps, L^p and Kosaki norms
`jordan`	Jordan product, quadratic representation and triple product; Jordan algebras generated by Hermitian matrices; spin factors

Table 3: Modules of `torch_vn_algebra`.

Caches. Scalars that are expensive or needed repeatedly are cached on the operator: the extreme eigenvalues (both from one call to the eigenvalue solver, without eigenvectors), the trace, the inverse and $|A|$. The eigendecomposition itself is kept only on request (`cache_spectrum()` per operator, or the context `caching()` for all): afterwards every function of the operator, for instance e^{-itH} for many t, costs two matrix products. The cache is keyed by the identity and the version counter of the stored tensor, so an in-place change invalidates it. Without caching, a decomposition computed for a positivity check is kept for exactly one further use, the functional calculus that typically follows. The algebra caches its own constants on the device (block sizes, masks, identities, trace weights, index tensors of the sector groups), so no Python list is copied to the GPU inside a loop.

Where the speed comes from

1. *Batching.* Every operation acts on the whole batch $(B, C, k_{\max}, k_{\max})$ in one kernel call; a Monte Carlo study of 10^5 samples is one tensor program, not 10^5 calls.
2. *One call per block size.* Decompositions are applied to the stacked active blocks of all sectors of equal size, one batched call per distinct size (`_size_groups`); contiguous sectors are selected as views, and when no sector is padded the result needs no scatter.
3. *One solver for everything.* Spectra, $f(A)$, $|A|$, singular values, polar decompositions and normal operators all go through the batched Hermitian eigensolver (Section 3, functional calculus), which is fast and not limited in size on GPUs, unlike batched SVD.

4. *Eigenvalues without eigenvectors* where only the spectrum is needed (extreme eigenvalues, contrasts, entropies of spectra).
5. *Direct formulas* instead of products, as above.
6. *No hidden synchronisation.* Inner loops do not call `.item()` or branch on device values; validations, ETA probes and convergence checks synchronise once per call, which keeps GPU queues full.
7. *Sparse where the structure is sparse.* Sector Hamiltonians of spin chains are CSR matrices for the batched Lanczos and Krylov solvers (Section 7). Jumps that map basis vectors to multiples of basis vectors (spin flips $S_i^{\pm}$, S_i^z, particle loss) are stored as `SectorJump`: a target sector per sector, a target position and a coefficient per basis vector. The Lindblad solvers then apply $L\rho L^*$ as a weighted permutation of the entries of ρ, the dual L^*ZL as a gather, and L^*L as a diagonal, in $O(\sum_c k_c^2)$ operations, against $O(C^2k_{\max}^3)$ and $C^2k_{\max}^2$ stored entries for the same jump as a dense map between sectors. A batch of coefficients gives one jump per sample, so a sweep over noise rates is one batched evolution.
8. *Constant generators are prepared once.* The Lindblad time steppers build $e^{-ihH_{\text{eff}}/2}$, the normalisation of the completely positive splitting, or $e^{h\mathcal{L}}$ once per step size and reuse them for all steps.

Table 4 measures several of these decisions by computing the same result with and without them.

mechanism	computation	library [ms]	without [ms]	speed-up
run scripts/architecture_ bench.py				

Table 4: The same result with and without one design decision (best of several runs, double precision, CPU with several threads; `scripts/architecture_bench.py`). "Without": eigenvalues by `eigvalsh` instead of the known spectrum; `eigh` instead of `eigvalsh`; one decomposition per time instead of the spectral cache; a Python loop over sectors; forming ρA; dense jumps between sectors. For the last row the eight jumps S_i^- take – MiB as dense maps and – MiB as `SectorJump`s, and the two evolutions agree to round-off.

Random unitaries. Haar unitaries on $U(n)$ and $O(n)$ are obtained from the QR decomposition of a Ginibre matrix with i.i.d. complex (respectively real) Gaussian entries after fixing the phases (signs) of $\operatorname{diag} R$ [14]; $SU(n)$ and $SO(n)$ by multiplying with $\det^{-1/n}$ or by a sign flip of one column; the circular orthogonal and symplectic ensembles as W^TW and W^RW with W Haar on $U(n)$ (n even for W^RW, W^R the quaternion dual) [15]. Section 5 checks all of them.

Functional calculus. The module `functional` provides $f(A)$ at four levels of generality, all built on batched kernels, one call per distinct block size [50]. (1) For self-adjoint A, $f(A) = Vf(\Lambda)V^*$ from the Hermitian eigensolver, for any function on the real spectrum. (2) For normal $A = H + iK$ (unitaries), H and K commute, and the eigenvectors of the Hermitian matrix $H+\alpha K$ diagonalise A for all but finitely many α; the library tries a few irrational values, refines the eigenvectors of nearly tied values with a second coefficient and corrects the remaining residual to first order, so normal operators also need only the Hermitian eigensolver (in single precision this decomposition is computed in double precision). (3) Singular values, $|A|$, the pseudo-inverse and the polar decomposition come from the Hermitian eigensolver of the dilation $\left(\begin{smallmatrix} 0 & A \\ A^* & 0 \end{smallmatrix}\right)$, whose eigenvalues are $\pm s_i$: batched SVD on GPUs is limited to blocks of size at most 32, while the batched Hermitian eigensolver is not. (4) For arbitrary

A and f holomorphic near the spectrum, $f(A) = (2\pi i)^{-1} \oint f(z)(z - A)^{-1} dz$ is evaluated by the trapezoidal rule on a circle [51], i.e. by batched LU solves, on a circle just around the spectrum, with the number of nodes doubled until two results agree; the error estimate includes the rounding error of cancelling terms, and an unconverged result raises an error instead of returning a wrong matrix. An opt-in cache keeps the eigendecomposition of an operator, so that many functions of it (for example e^{-itH} for many t) cost matrix products only.

Differentiation. All operations are differentiable. Gradients propagated naively through eigenvectors contain factors $1/(\lambda_i - \lambda_j)$ and fail at degenerate spectra, which are typical here (projections, symmetric operators, extremal operators). The functional calculus $f(A) = V f(\Lambda) V^*$ therefore has its own backward pass: by the Daleckii–Krein formula [52, 53], the derivative of f at a self-adjoint A is $dF = V\,(L \circ V^* dA\, V)\, V^*$ with the Loewner matrix of divided differences $L_{ij} = (f(\lambda_i) - f(\lambda_j))/(\lambda_i - \lambda_j)$, replaced by f' for coinciding eigenvalues, which is finite whenever f is differentiable on the spectrum (for $f \in C^1$ it is the Fréchet derivative). Backward passes through the eigenvectors themselves stay finite at degenerate spectra only for perturbations or losses that do not mix a degenerate eigenspace [54]; they then miss the entries $f'(\lambda)$ of L inside the degenerate blocks, which the Daleckii–Krein formula includes. Gradients with respect to parameters of f (a time in e^{-itA}) are propagated as usual. The same applies to $\sqrt{A}$, $\log A$, A^p, and to $|A|$ and the inverse of self-adjoint A, which use the Hermitian eigensolver instead of the SVD. For normal A the same formula with complex divided differences is the derivative of the holomorphic calculus, for holomorphic f. $|A|$ of a general A is differentiated as the square root of A^*A, with $L_{ij} = 1/(s_i + s_j)$, and the pseudo-inverse by the constant-rank formula; the contour integral is differentiated through the solves. Only gradients through the eigenvectors returned by `eigh` keep the factors $1/(\lambda_i - \lambda_j)$; the parametrisations of Section 6 avoid them. `Operator.eigenvalues()` computes spectra without forming eigenvectors, which also saves time and memory.

GPUs, tensor cores and TF32. The library computes in FP32 or FP64 (real or complex); it does not use half precision. GPU generations differ strongly in what this costs. Tensor cores, absent on the P100 used for the first version, accelerate matrix products on Volta and later GPUs; from Ampere (A100) on, FP32 products may be executed in the TF32 format with a 10-bit mantissa, which is fast but accurate only to about 10^{-3} relative. PyTorch (since 1.12) keeps true FP32 for matrix products by default, and the library does not change this setting; the paper scripts expose a `-tf32` switch to measure the effect and record the setting in every table. TF32 is acceptable where the statistical error dominates (moments and histograms of many samples) and not where small differences matter (eigenvalues near degeneracies, the validation tables, trace inequalities near equality). Eigen- and singular-value solvers do not use tensor cores. FP64 throughput is half of FP32 on data-centre GPUs (P100, V100, A100, H100), the latter two with FP64 tensor cores, but only 1/32–1/64 on consumer GPUs. We therefore recommend a data-centre GPU for double-precision work. The full-size runs of this paper were made on a consumer GPU (the tables state the hardware); the double-precision timings there are therefore pessimistic for the GPU.

Number field and precision

Every algebra is created with two independent switches, `complex_valued` (real or complex matrices) and `precision` (`'single'` or `'double'`), giving the storage types `float32`, `float64`, `complex64` and `complex128`; everything built on the algebra (random operators, states, channels, chains, parametrisations) inherits them. `alg.like(complex_valued=..., precision=...)` creates the same sector structure with other numerics and `op.cast(alg2)` moves an operator there, e.g. to explore in single

precision and re-evaluate the interesting cases in double precision.

The field is part of the model. Choosing real matrices is not only a matter of speed: the symmetry group and the random ensembles change. Haar matrices of a real algebra are orthogonal, so moments, level statistics and extremal values refer to $O(n)$ instead of $U(n)$ (e.g. $\mathbb{E}|U_{11}|^4 = 3/(n(n+2))$ instead of $2/(n(n+1))$, Table 8); random real symmetric and complex Hermitian matrices have different level statistics, with mean spacing ratios 0.5307 and 0.5996 for GOE and GUE (large-n numerical values of [24, Table I]; the 3×3 surmise gives $4 - 2\sqrt{3} \approx 0.5359$ and $2\sqrt{3}/\pi - \frac{1}{2} \approx 0.6027$) (our samples: 0.5313 and 0.5993). Real arithmetic is *exact* (loses nothing) when the problem is real: spectra and eigenvectors of real symmetric matrices, ground states and their entanglement for Hamiltonians without complex hoppings, statistics of time-reversal-invariant systems, and also some extremal problems — for real $X, Y \geq 0$ the supremum of $|\operatorname{Tr}(XUY)|$ over unitaries is attained by a real orthogonal matrix (the polar decomposition of the real matrix YX is real). Complex matrices are *required* for unitary time evolution e^{-iHt} and Lindblad dynamics with a Hamiltonian (the library refuses to run them in a real algebra), for Haar unitaries, CUE/CSE and $SU(n)$, for random pure states and quantities such as Page's entropy, and for systems without an antiunitary symmetry squaring to $+1$ (orbital fluxes, complex couplings).

Precision. Single precision resolves about 10^{-7} relative to the largest entry. It is sufficient for Monte Carlo statistics whose statistical error is much larger (moments, histograms, spacing ratios with 10^3–10^5 samples) and for the first, exploratory pass of an optimisation. Double precision is needed when the quantity of interest is small compared with the operators: an inequality checked near equality, small spectral gaps, entropies and logarithms of nearly singular states, contrasts close to 1, comparisons with exact results beyond six digits, long RK4 integrations and optimisation to high relative accuracy. Table 6 shows how drastic the difference can be: the von Neumann entropy of an almost pure state with eigenvalues of order 10^{-9} is wrong by more than 100 % in single precision, and the round-off floor of a trace inequality that holds with equality is several orders of magnitude higher in single than in double precision (Table 6).

Cost. An entry takes 4, 8, 8 and 16 bytes for `float32`, `float64`, `complex64` and `complex128`, so memory, and the batch that fits, changes by a factor up to four (`alg.bytes_per_operator(B)` reports it). A complex multiply–add costs about four real ones, which bounds the slowdown of complex matrix products; for eigen- and singular-value solvers, and for double against single precision on CPUs, the measured ratios are smaller (Table 5). On GPUs double precision depends on the hardware: data-centre GPUs execute FP64 at half the FP32 rate, consumer GPUs at 1/32–1/64 of it, so that double precision there can be an order of magnitude slower. Table 5 gives the measured ratios for the current run; the recommendation that follows is to use the smallest type that is exact for the model (real if the problem is real) and the precision that the size of the effect requires.

Safeguards. Batched computations make it easy to request hours of work or more memory than the device has. The `cost` module therefore (a) times the first few items of every large batched eigen-, singular-value or QR decomposition and matrix exponential and issues a warning with the expected total time when it exceeds a threshold (10 s by default), (b) compares the estimated peak memory of large allocations with the free memory of the device, warning above 50 % and raising an `InsufficientMemoryError` above 90 %, and (c) after the second step of a time-stepping or optimisation loop, warns with the expected duration if it exceeds the same threshold; a progress bar is optional. All thresholds can be changed, and the checks can be switched off locally.

type	B/entry	MB/batch	generate	eigh f(A)	SVD |A|	channel	expm	opt. step
`float32`	4	16	1	1	1	1	1	1
`float64`	8	32	1.2	7.7	21	3.7	2.1	3.4
`complex64`	8	32	1.1	1.6	1.8	1.8	1.5	1.8
`complex128`	16	64	3.2	11	65	13	5.4	11

Table 5: Bytes per entry, memory of one batch [MB], and time relative to `float32` for $C = 16$ sectors of size $k = 32$ and a batch of 256 operators (full run on NVIDIA GeForce RTX 5090; float32 times per sample: generate: 918 μs, eigh f(A): 7.24 μs, SVD |A|: 11.4 μs, channel: 1.83 μs, expm: 4.96 μs, opt. step: 70.9 μs).

type	$\max \|UU^* - 1\|$	$\|\sqrt{A}^2 - A\|/\|A\|$	rel. error $S(\rho)$, $\rho \approx$ pure	$\|z\|$ at equality
`float32`	6.6×10^{-7}	2.0×10^{-6}	2.0×10^{0}	5.3×10^{-4}
`float64`	1.3×10^{-15}	4.0×10^{-15}	6.6×10^{-9}	1.4×10^{-12}
`complex64`	8.3×10^{-7}	2.3×10^{-6}	2.3×10^{0}	2.0×10^{-4}
`complex128`	1.6×10^{-15}	4.5×10^{-15}	8.9×10^{-9}	1.8×10^{-12}

Table 6: Accuracy of the same computations in the four storage types: Haar unitaries ($n = 64$), SVD square root, entropy of $\rho = (1-\varepsilon)|0\rangle\langle 0| + \varepsilon\, 1/n$ with $\varepsilon = 10^{-7}$, and the round-off floor of $z = \mathrm{Tr}(Y|X|Y) - \mathrm{Tr}\,|YXY|$ for commuting X, Y (exactly 0); full run on NVIDIA GeForce RTX 5090.

4 Usage by example

The repository contains short complete programs in `man/article/listings`; the test suite runs them, so they stay in sync with the code. Each runs on a CPU in under half a minute and uses a GPU when available. Table 7 lists the nine introductory examples; the programs of later sections are described where they are used.

Some of the examples check exact statements. In `ex3`, the Kraus operators $\sqrt{1-gN}$, $\sqrt{g}\,a_1$, $\sqrt{g}\,a_2$ ($0 \le g \le \frac{1}{2}$) map sector N to N and $N-1$, and the classical part is the transition matrix of a pure death process (columns: from $N = 0, 1, 2$; rows: to $N = 0, 1, 2$), $T = \left(\begin{smallmatrix} 1 & g & 0 \\ 0 & 1-g & 2g \\ 0 & 0 & 1-2g \end{smallmatrix}\right)$; a unital positive map cannot increase the Michelson contrast, since $\lambda_{\min} 1 \le A \le \lambda_{\max} 1$ implies the same bounds for $\Phi(A)$, while the non-unital loss channel increases it for most samples. In `ex7` the ground state moves through the sectors until the chain is fully polarised above the saturation field $h_s = 2J$ (for our sign $+\sum_i h_i S_i^z$ the all-down state $N = 0$; the last crossing is with the one-magnon level at $k = \pi$). In `ex9` the mixer $H_M = \frac{1}{4}\sum_{\langle ij\rangle}(X_iX_j + Y_iY_j)$ keeps the Hamming weight [38], so the register carries the algebra $\bigoplus_{N=0}^{n} M_{\binom{n}{N}}$; amplitude damping ($S_i^-$, rate γ_1) is a jump between sectors, dephasing (S_i^z, rate γ_2) a jump inside them, and reading out the weight measures a central projection. With damping only, the post-selected fidelity is exactly one and the acceptance probability is $e^{-w\gamma_1 T}$, because the no-jump evolution is generated by $H - \frac{i}{2}\gamma_1 N$ and N is central; for the same reason the post-selected state does not depend on γ_1 even with dephasing. For $|111000\rangle$, $T = 2$ and $\gamma_1 = \gamma_2 = 0.02$ the program gives the acceptance probability 0.8869, the fidelity 0.8629 and 0.9730 after post-selection.

5 Validation against exact results

Table 8 compares Monte Carlo estimates produced with the library (double precision) with exact values: fourth moments of Haar unitaries and orthogonal matrices; the Weingarten-calculus value

program	what it shows	key API
`listings/ex1_random_operators.py`	10^4 positive operators in $M_2 \oplus M_3 \oplus M_4$ with contrast 0.6 and Haar-random eigenvectors in every sector; the three traces and spectral functions	`TypeIAlgebra`, `operator_from_eigenvalues`, `michelson_contrast`, `Tr_blunt`, `tau_vN`, `apply_function`
`listings/ex2_states_measurement.py`	Lüders conditioning on a projective measurement moves all probability into one sector; expectation values do not depend on the trace that represents the state	`states.random_density_matrix`, `condition_on`, `sector_probabilities`, `density`
`listings/ex3_channels.py`	particle loss of two fermionic modes as a map between the sectors $N = 0, 1, 2$; unital and non-unital channels and the contrast	`InterSectorChannel.from_blocks`, `transition_matrix`, `random_mixed_unitary_channel`
`listings/ex4_dynamics.py`	spontaneous emission of a two-level atom by RK4 and by the exponential of the generator, against $P_e(0)e^{-\gamma t}$ and $\lvert\rho_{eg}(0)\rvert e^{-\gamma t/2}$	`dynamics.lindblad_evolve`, `dynamics.lindblad_channel`
`listings/ex5_page_entropy.py`	mean entanglement entropy of random pure states on $\mathbb{C}^m \otimes \mathbb{C}^n$ against Page's formula [16]	`tensor_product`, `partial_trace`, `states.von_neumann_entropy`
`listings/ex6_inequality_search.py`	Haar sampling against batched gradient ascent for $\sup_U(\lvert\operatorname{Tr}(XUY)\rvert - \operatorname{Tr}(XY)) = \lVert YX\rVert_1 - \operatorname{Tr}(XY)$ (Section 6)	`random_unitary_operator`, `expm`, `trace_norm`
`listings/ex7_spin_chain.py`	Heisenberg ring of ten spins: the sector of the ground state in a uniform field, the mean level-spacing ratio of 100 disorder realisations in one batch, entanglement entropies	`SpinChain`, `random_field_heisenberg`, `level_spacing_ratio`, `entanglement_entropy`
`listings/ex8_large_and_open_chains.py`	Lanczos ground state of a sector of dimension 12870, Krylov evolution of a domain wall, momentum sectors, particle loss through jumps between sectors	`xxz_sparse`, `krylov.ground_state`, `krylov.evolve`, `xxz_momentum`
`listings/ex9_qaoa_xy_mixer.py`	XY mixer on a ring of six qubits with amplitude damping, dephasing and post-selection on the Hamming weight	`SpinChain.xxz`, `lowering`, `algebra.central`, `condition_on`

Table 7: The introductory programs in `man/article/listings` of the repository.

$\mathbb{E}\,\tau\big((AUBU^*)^2\big) = -1/(n^2-1)$ for traceless involutions $A = B$ and Haar U in $M_n(\mathbb{C})$ [20]; Page's formula for $\langle S(\rho_A)\rangle$, conjectured in [16] and proved in [17, 18]; and the mean purity $(k+r)/(kr+1)$ of random states of M_k induced by an ancilla of dimension $r \le k$ (so of rank r), due to Lubkin [19] (see also [21, Eq. (3.20)]). The last column is the deviation in units of the standard error; for a correct sampler these should look like standard normal deviates. For the circular ensembles the Wigner surmise is only an approximation to the exact spacing distribution, so no z-score is given; Figure 1 shows the distributions, with L^1 distances 0.022, 0.007 and 0.006 for $\beta = 1, 2, 4$ from the surmise; level repulsion $P(s) \sim s^\beta$ is clearly visible. The spontaneous-emission example agrees with the analytic solution to 2.2×10^{-16} (exact exponential) and 2.1×10^{-8} (RK4).

Trace functionals. The three traces are checked on random non-Hermitian operators in $M_2 \oplus M_3 \oplus M_5$ (Table 9): normalisation on the identity ($\sum_c k_c$, C and 1), the tracial property $\operatorname{tr}(AB) = \operatorname{tr}(BA)$, invariance under unitaries of $\mathcal{M}$, and positivity $\operatorname{tr}(A^*A) > 0$ (faithfulness); the last column is the smallest ratio $\operatorname{tr}(A^*A)/\lVert A\rVert_F^2$ over the samples, which is bounded below by $\min_c w_c$. The relation $\mathrm{Tr}_{\mathrm{norm}} = C\,\tau_{\mathrm{vN}}$ holds to 9.0×10^{-16}. All three return values in the number field of the algebra (complex traces of non-Hermitian operators).

quantity	parameters	exact	Monte Carlo	std. error	z
$\mathbb{E}\lvert U_{11}\rvert^4$, $U(n)$	$n=2$	0.333333	0.333604	4.7×10^{-4}	+0.6
$\mathbb{E}\lvert U_{11}\rvert^2$, $U(n)$	$n=2$	0.5	0.500277	4.6×10^{-4}	+0.6
$\mathbb{E}\lvert U_{11}\rvert^4$, $U(n)$	$n=4$	0.1	0.10028	2.2×10^{-4}	+1.3
$\mathbb{E}\lvert U_{11}\rvert^4$, $U(n)$	$n=8$	0.0277778	0.0277966	7.5×10^{-5}	+0.3
$\mathbb{E}\lvert U_{11}\rvert^4$, $U(n)$	$n=16$	0.00735294	0.00735803	2.3×10^{-5}	+0.2
$\mathbb{E}\lvert U_{11}\rvert^4$, $U(n)$	$n=32$	0.00189394	0.00189625	6.2×10^{-6}	+0.4
$\mathbb{E}\lvert U_{11}\rvert^2$, $U(n)$	$n=32$	0.03125	0.0312403	4.8×10^{-5}	-0.2
$\mathbb{E}\lvert U_{11}\rvert^4$, $O(n)$	$n=2$	0.375	0.375106	5.8×10^{-4}	+0.2
$\mathbb{E}\lvert U_{11}\rvert^2$, $O(n)$	$n=2$	0.5	0.500155	5.6×10^{-4}	+0.3
$\mathbb{E}\lvert U_{11}\rvert^4$, $O(n)$	$n=4$	0.125	0.125085	3.1×10^{-4}	+0.3
$\mathbb{E}\lvert U_{11}\rvert^4$, $O(n)$	$n=8$	0.0375	0.0376573	1.3×10^{-4}	+1.2
$\mathbb{E}\lvert U_{11}\rvert^4$, $O(n)$	$n=16$	0.0104167	0.0104046	4.3×10^{-5}	-0.3
$\mathbb{E}\lvert U_{11}\rvert^4$, $O(n)$	$n=32$	0.00275735	0.0027558	1.2×10^{-5}	-0.1
$\mathbb{E}\lvert U_{11}\rvert^2$, $O(n)$	$n=32$	0.03125	0.0312731	6.7×10^{-5}	+0.3
$\mathbb{E}\,\tau((AUBU^*)^2)$	$n=4$	-0.0666667	-0.0661548	6.3×10^{-4}	+0.8
$\mathbb{E}\,\tau((AUBU^*)^2)$	$n=8$	-0.015873	-0.0159874	2.8×10^{-4}	-0.4
$\mathbb{E}\,\tau((AUBU^*)^2)$	$n=16$	-0.00392157	-0.00414221	1.4×10^{-4}	-1.6
$\langle s^2\rangle$, COE	$n=64$	1.27324	1.28582	3.7×10^{-4}	–
$\langle s^2\rangle$, CUE	$n=64$	1.1781	1.18024	2.4×10^{-4}	–
$\langle s^2\rangle$, CSE	$n=64$	1.10447	1.10392	2.0×10^{-4}	–
$\langle S(\rho_A)\rangle$	$m=2, n=16$	0.646516	0.646516	1.6×10^{-4}	0.0
$\langle S(\rho_A)\rangle$	$m=8, n=16$	1.83367	1.83388	1.7×10^{-4}	+1.2
$\langle S(\rho_A)\rangle$	$m=16, n=16$	2.27487	2.27495	1.4×10^{-4}	+0.6
$\mathbb{E}\,\mathrm{Tr}\,\rho^2$	$k=4, r=2$	0.666667	0.666312	3.2×10^{-4}	-1.1
$\mathbb{E}\,\mathrm{Tr}\,\rho^2$	$k=4, r=4$	0.470588	0.470523	2.1×10^{-4}	-0.3
$\mathbb{E}\,\mathrm{Tr}\,\rho^2$	$k=16, r=4$	0.307692	0.307726	6.4×10^{-5}	+0.5
$\mathbb{E}\,\mathrm{Tr}\,\rho^2$	$k=16, r=16$	0.124514	0.124527	1.7×10^{-5}	+0.8

Table 8: Monte Carlo estimates versus exact values (for $\langle s^2\rangle$ the Wigner surmise, an approximation, hence no z); samples: 400000 per Haar moment, 400000 per Weingarten row, 20000 matrices per circular ensemble, 50000 states per Page row, 100000 per purity row (full run on NVIDIA GeForce RTX 5090, PyTorch 2.14.1+cu130).

6 From sampling to optimisation

The first version of this work [4], and before it [9, Sect. 5, Figs. 1–3] for a single block $M_n(\mathbb{R})$ with $n \le 5$, studied three trace functionals by Monte Carlo:

$$\begin{aligned}
&\text{Exp. 1:} \quad z = |\,\mathrm{Tr}(XUY)| - \mathrm{Tr}(XY), && X, Y \ge 0,\ U \in \mathcal{M} \text{ unitary},\\
&\text{Exp. 2:} \quad z = \mathrm{Tr}\,|XY| - \mathrm{Tr}(|X|\,|Y|), && X = UX_0,\ Y = VY_0,\ X_0, Y_0 \ge 0,\ U, V \in \mathcal{M} \text{ unitary},\\
&\text{Exp. 3:} \quad z = \mathrm{Tr}(Y|X|Y) - \mathrm{Tr}\,|YXY|, && X = X^*,\ Y \ge 0,
\end{aligned}$$

with $\mathrm{Tr} = \mathrm{Tr}_{\mathrm{blunt}}$, operators normalised to $\lambda_{\max} = 1$, and the contrasts of the operators as parameters (for the self-adjoint X of Exp. 3 and the factors of Exp. 2, the contrasts of $|X|, |Y|$). Two observations motivate a different approach.

Sampling misses the extremes. For fixed X, Y the supremum in Exp. 1 is known exactly, $\sup_U z = \|YX\|_1 - \mathrm{Tr}(XY) > 0$. Table 10 and Figure 2 show that the best of 1 000 000 Haar samples falls short of the supremum for every k, misses the violation for all pairs from $k = 4$ on, and moves further away from it as k grows, while batched gradient ascent (128 starts, 500 steps)

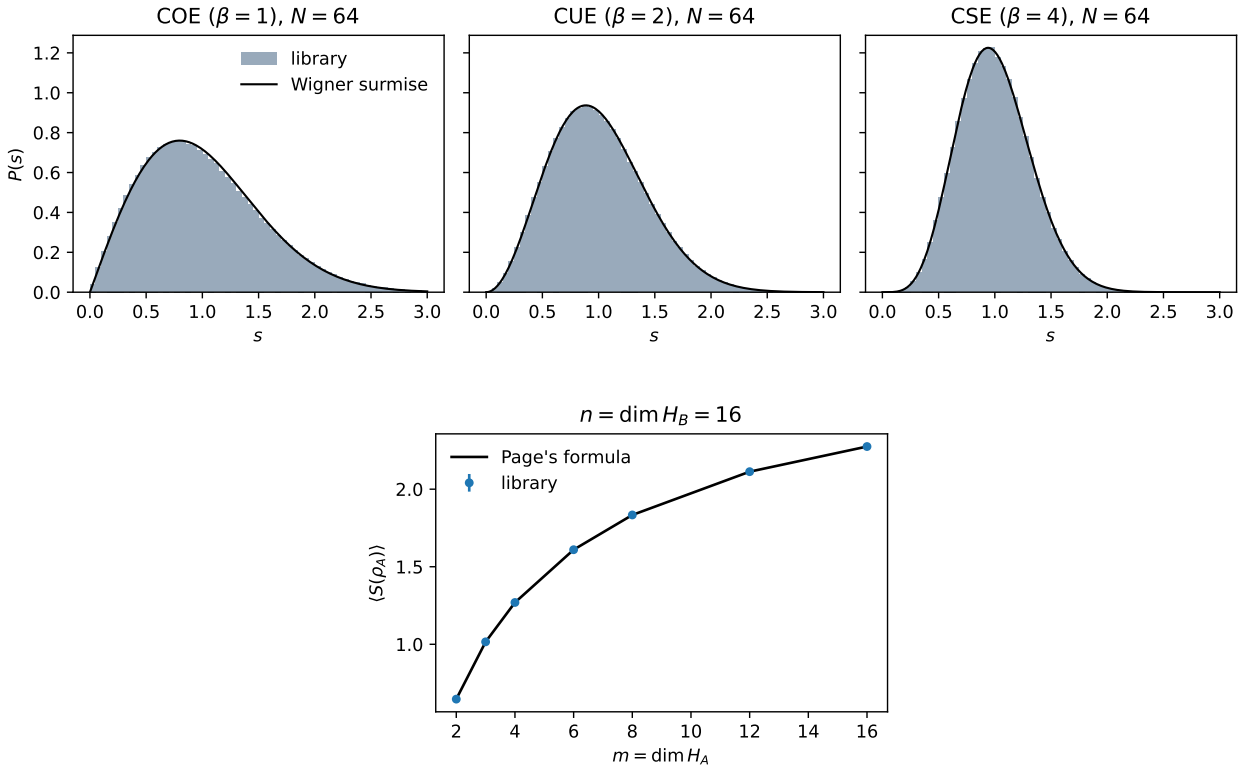


Figure 1: Top: nearest-neighbour spacings of COE, CUE and CSE eigenphases generated by the library ($n = 64$, 20000 matrices per ensemble), against the Wigner surmise. Bottom: mean entanglement entropy of random pure states against Page's formula (50000 states per point) (error bars: two standard errors).

finds it to a relative error of at most 3×10^{-5} (Table 10). This is concentration of measure: the unitaries close to the maximiser form a set whose Haar measure decays rapidly with the dimension. This search is done in the complex algebra $M_k(\mathbb{C})$ ($C = 1$, eigenvalues of X, Y uniform in $[0.2, 1.2]$, double precision); the same concentration effect explains the absence of violations in the (real) Monte Carlo samples for $k = 16$ reported in [4]: it is a property of the sampling, not of the inequality.

Constrained optimisation. The natural objects are envelopes such as

$$\overline{z}(\delta) = \sup\{z : \Delta(X) = \delta\}, \qquad \underline{z}(\delta) = \inf\{z : \Delta(X) = \delta\},$$

the supremum being taken over all remaining variables. The module `optimize` parametrises the constraint set exactly: the spectrum is $\lambda_{\max} = 1$, $\lambda_{\min} = (1-\delta)/(1+\delta)$ and the other eigenvalues in between, as $\lambda_{\min} + (1-\lambda_{\min})(1+\sin\theta)/2$ (unlike a sigmoid, this reaches both ends at finite θ; extremal spectra often have all eigenvalues at the ends); the two eigenvalue positions that carry $\lambda_{\max}$ and $\lambda_{\min}$ are drawn at random per start, in the same sector or in different ones, so that the starts together cover a dense subset of the constraint set, the eigenvectors are $U = U_0 e^{G - G^*}$ with a Haar-random U_0 per start and a chart that is re-centred after every round. Discrete variables, such as the signs of the eigenvalues of a self-adjoint X (Exp. 3), are handled between the gradient rounds: the number of negative eigenvalues is drawn per sector (for half of the starts as one fraction common to all sectors), and between the gradient rounds single signs are flipped where this improves z. Every iterate is an

trace	error of tr(1)	$\max\lvert\operatorname{tr}(AB)-\operatorname{tr}(BA)\rvert$	$\max\lvert\operatorname{tr}(UAU^*)-\operatorname{tr}(A)\rvert$	$\min\operatorname{tr}(A^*A)/\lVert A\rVert_F^2$
$\mathrm{Tr}_{\mathrm{blunt}}$	0	4.0×10^{-15}	5.9×10^{-15}	1.000
$\mathrm{Tr}_{\mathrm{norm}}$	0	1.4×10^{-15}	1.9×10^{-15}	0.223
τ_{vN}	0	4.4×10^{-16}	6.1×10^{-16}	0.074

Table 9: Identities of the three trace functionals, 2000 random operators in double precision; full run on NVIDIA GeForce RTX 5090.

k	violation found		$\sup z/\operatorname{Tr}XY$	best sample	ascent rel. error		time [s]	
	sampling	ascent		$z/\operatorname{Tr}XY$	median	max	sampl.	ascent
2	14/16	16/16	7.1×10^{-4}	5.6×10^{-4}	1.2×10^{-8}	5.4×10^{-6}	64	2.8
4	0/16	16/16	2.3×10^{-3}	-0.12	4.4×10^{-8}	3.1×10^{-6}	66	2.8
8	0/16	16/16	5.6×10^{-3}	-0.48	2.9×10^{-7}	2.0×10^{-6}	70	2.8
16	0/16	16/16	6.7×10^{-3}	-0.72	2.4×10^{-6}	3.9×10^{-6}	88	2.7
32	0/16	16/16	8.9×10^{-3}	-0.86	7.4×10^{-6}	1.6×10^{-5}	1.8e+02	6.6
64	0/16	16/16	7.0×10^{-3}	-0.93	2.0×10^{-5}	2.8×10^{-5}	2.7e+02	11

Table 10: Exp. 1 for fixed random X, Y in $M_k(\mathbb{C})$ (eigenvalues uniform in $[0.2, 1.2]$, double precision): Haar sampling versus gradient ascent (medians over 16 pairs; full run on NVIDIA GeForce RTX 5090).

admissible point, so the best value seen per start is kept. Every grid point and every start is one element of the batch, so each envelope (supremum or infimum along one contrast) is computed in one batched optimisation.

Two implementation details turned out to matter. (a) In a real algebra a chart $U_0e^{G-G^*}$ never leaves the connected component of U_0, and an extremiser with determinant $+1$ in every sector is reached from a Haar-random U_0 only with probability 2^{-C}; half of the starts therefore begin at a structured point (`init`, `spectrum_init`: diagonal X, Y rotated in pairs of eigenvectors of X, two-level spectra). With Haar-random starts only, a first run for $C = 32$ sectors of size 2 fell short of the true envelopes by up to 35%; with the structured starts it reaches them to a relative 10^{-5}–10^{-3}. (b) Where the objective has a kink at the maximiser (a trace norm at a degenerate point), the remaining gap decreases with the step size and is smoothed only on the scale of the smallest admissible eigenvalue. Applied to the three trace functionals of [4, 9], the optimiser located their envelopes for sectors of every size; the closed forms it suggested are proved, and compared with sampling, in the companion paper [1].

6.1 Rediscovering the data-processing thresholds of the Rényi divergences

The envelopes above were first found numerically and proved afterwards [1]. A further test of the method, on an inequality whose range of validity is known exactly, is to ask it where that range ends, without telling it. For states ρ, σ on $\mathcal{M}$ (densities with respect to $\mathrm{Tr}_{\mathrm{blunt}}$) and $\alpha \in (0,1)\cup(1,\infty)$ the Petz [55] and the sandwiched [58, 59] Rényi divergences are

$$D_\alpha(\rho\|\sigma) = \frac{1}{\alpha-1}\log\operatorname{Tr}\bigl(\rho^\alpha\sigma^{1-\alpha}\bigr), \qquad \widetilde{D}_\alpha(\rho\|\sigma) = \frac{1}{\alpha-1}\log\operatorname{Tr}\bigl(\sigma^{\frac{1-\alpha}{2\alpha}}\,\rho\,\sigma^{\frac{1-\alpha}{2\alpha}}\bigr)^\alpha.$$

Both tend to the Umegaki relative entropy $D(\rho\|\sigma) = \operatorname{Tr}\rho(\log\rho - \log\sigma)$ as $\alpha \to 1$; $\widetilde{D}_\alpha \le D_\alpha$, $\widetilde{D}_{1/2} = -\log F(\rho,\sigma)$ with the fidelity $F = \|\sqrt{\rho}\sqrt{\sigma}\|_1^2$, and $\widetilde{D}_\alpha \to D_{\max}$ as $\alpha \to \infty$. The module `divergences` evaluates them, batched and differentiable (Daleckii–Krein gradients), with exact support conventions (powers are taken on the numerical support, and the divergence is $+\infty$ for $\alpha > 1$ when $\operatorname{supp}\rho \not\le$

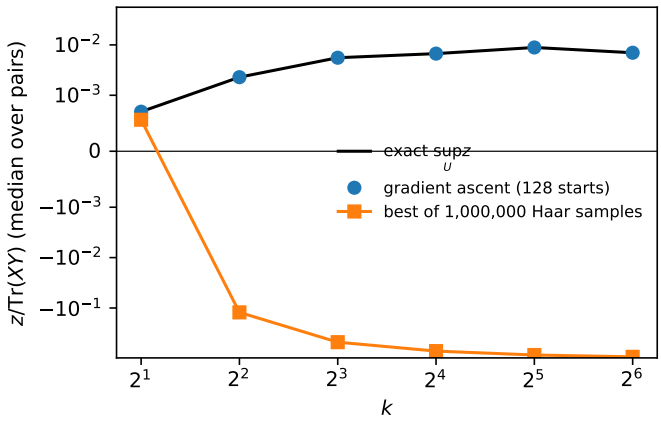


Figure 2: Best z found by sampling and by gradient ascent, relative to $\mathrm{Tr}(XY)$, against the exact supremum (symmetric log scale).

$\operatorname{supp}\sigma$), as logarithms of sums over sectors, so that large α neither overflows nor underflows. On $\mathcal{M}$ they split into a classical part and the conditional divergences of the sectors, and they do not depend on the trace used to represent the states (the central weights cancel); the tests compare them with dense references and check these properties.

The data-processing inequality $D(\Phi(\rho)\|\Phi(\sigma)) \le D(\rho\|\sigma)$ for all channels Φ holds for the Petz divergences for $\alpha \in (0,1)\cup(1,2]$ [55] and for the sandwiched ones for $\alpha \ge \frac{1}{2}$; for the latter it was proved for $\alpha \in (1,2]$ in [58, 59], for all $\alpha > 1$ in [61], and for all $\alpha \ge \frac{1}{2}$ in [60]. Outside these ranges it fails in general: for the Petz divergences with $\alpha > 2$ the trace functional is neither convex nor concave in σ [58], and for the sandwiched ones with $\alpha < \frac{1}{2}$ numerical counterexamples were reported in [58], and by [62] it fails, for a suitable measurement, for every pair of non-commuting states. Both ranges are the cases $z = 1$ and $z = \alpha$ of the data-processing range of the α-z Rényi divergences, which Zhang determined completely [63]. For the relative entropy, equality characterises the channels that can be reversed on ρ and σ, and the reversal is the Petz recovery map (for subalgebras and conditional expectations [56], for channels [57]) $R_{\sigma,\Phi}(X) = \sigma^{1/2}\Phi^*\big(\Phi(\sigma)^{-1/2}X\Phi(\sigma)^{-1/2}\big)\sigma^{1/2}$, which satisfies $R_{\sigma,\Phi}(\Phi(\sigma)) = \sigma$. The module provides it for channels within and between sectors, together with its rotated versions and the universal recovery map R_{univ} of [64], for which $D(\rho\|\sigma) - D(\Phi(\rho)\|\Phi(\sigma)) \ge -\log F(\rho, R_{\mathrm{univ}}(\Phi(\rho)))$ whenever $\operatorname{supp}\rho \le \operatorname{supp}\sigma$ ([64, Remark 2.2], written there with the root fidelity $\sqrt{F}$ and the factor $-2\log$); the tests check the inequalities inside their ranges on random states and channels, Petz's equality condition and the recovery bound.

`scripts/renyi_threshold.py` maximises the violation

$$v(\alpha) = D(\Phi(\rho)\|\Phi(\sigma)) - D(\rho\|\sigma)$$

for both families on a grid of 31 values $\alpha \in [0.2, 3]$, over states ρ, σ on M_d and over two kinds of channels: the complete dephasing (the conditional expectation onto the diagonal subalgebra) and all channels with a d-dimensional environment, given by Stinespring isometries $V = Z(Z^*Z)^{-1/2}$ with a free complex Z. All values of α and all starts form one batch (α is given per sample), optimised with the batched multistart ascent of Section 6. The states are restricted to $\rho, \sigma \ge \varepsilon/d$; this keeps the supremum finite and the sandwiched divergence resolved in double precision for $\alpha \ge 0.2$. The bracket between the last violating and the first non-violating grid point is then refined by searches at interior points, started from random points and from perturbations of the best violating point. Every reported violation is attained at an explicit point (two states and the Kraus operators of a channel), and v is

recomputed there independently in mpmath with 50 significant digits, after re-orthonormalising the Kraus operators so that the point is an exactly trace-preserving channel; it counts as a violation only if the certified value exceeds a tolerance far above the error of the certification. A certified violation therefore proves that the inequality fails at that α; the absence of a violation at the other end of a bracket is evidence only, since a local search can miss small violations. (Results not generated: run `scripts/renyi_threshold.py`.) Table 11 and Figure 3 show $v(\alpha)$ on the grid.

	sandwiched $\widetilde{D}_\alpha$		Petz D_α	
α	dephasing	channels	dephasing	channels
run scripts/renyi_ threshold.py				

Table 11: Largest certified violation $v(\alpha)$ of the data-processing inequality on M_2, for the complete dephasing and for all channels with a two-dimensional environment ("–": no violation above the tolerance found). The theorems guarantee $v \le 0$ for $\alpha \ge \frac{1}{2}$ (sandwiched) and $\alpha \le 2$ (Petz).

figure renyi_threshold.pdf not generated

Figure 3: Largest certified violation $v(\alpha)$ (logarithmic axis) for the sandwiched (left) and the Petz (right) divergences, for the complete dephasing and for channels with a d-dimensional environment (solid: $d = 2$, dotted: $d = 3$); open symbols on the bottom line: no violation above the tolerance (dotted horizontal line) found. Dashed: the known thresholds; shaded: the range in which the data-processing inequality is a theorem.

7 Symmetric tensor products and spin chains

The basis of `SpinChain` is, by construction, that of the iterated fused product of single sites: the tests check that S_i^z built by the chain coincides with $S_i^z \otimes 1$ formed with `kron` in the fused product of the algebra of `SpinChain(L-1)` with a single site. Beyond that, integrable chains provide exact results for every part of the module (Table 13, Figure 4; script `scripts/chains.py`):

- *Free fermions.* By the Jordan–Wigner transformation the open XX chain ($\Delta = 0$) is a free-fermion model with single-particle energies $\varepsilon_k = J \cos \frac{\pi k}{L+1}$, so the spectrum of sector N consists of all sums of N distinct ε_k. For $L = 12$ all 2^L levels agree to 7.1×10^{-15}.

- *Heisenberg ring.* Ground-state energies agree with exact values (Bethe ansatz for $L = 4$, independent exact diagonalisation for the others, rounded to nine digits) to the nine digits of the table; E_0/L approaches the Bethe-ansatz value $\frac{1}{4} - \ln 2 \approx -0.4431$. Every level of sector $N < L/2$ reappears in sector $N + 1$ (SU(2) multiplets), up to 2.9×10^{-15}; Section 8 uses this symmetry to reduce the blocks further.

- *Dynamics.* The melting of a domain wall $|\uparrow \cdots \uparrow\downarrow \cdots \downarrow\rangle$ in the open XX chain, evolved in the sector $N = L/2$ (dimension 12870 for $L = 16$), reproduces the single-particle prediction $\langle n_i(t)\rangle = [e^{iht} C(0) e^{-iht}]_{ii}$, with h the single-particle hopping matrix and $C_{ij} = \langle c_i^\dagger c_j\rangle$ the correlation matrix, to 4.9×10^{-15}.

- *Entanglement.* The entropy of a block of n sites of the XX ground state, obtained with `reduced_state`, agrees with Peschel's formula $S = -\sum_\nu [\nu \ln \nu + (1-\nu)\ln(1-\nu)]$ over the eigenvalues ν of the restricted correlation matrix [22], including the parity oscillations of open chains, to 3.8×10^{-12}.

- *Disorder.* For the Heisenberg chain with random fields $h_i \in [-W, W]$ the mean ratio of consecutive level spacings in the middle of the spectrum crosses over from the GOE value 0.5307 to the Poisson value $2\ln 2 - 1 \approx 0.3863$ [23, 24] as W grows; at these sizes this is a finite-size crossover, the standard spectral diagnostic used in studies of many-body localisation. Here the batch dimension holds the disorder realisations (400 per point).

- *Momentum.* For periodic chains the basis $|a, k\rangle = N_a^{-1/2} \sum_{j=0}^{L-1} e^{-ikj} T^j |a\rangle$ over orbit representatives a of the translation T (orbit length R_a, $N_a = L^2/R_a$; a contributes to momentum k iff $kR_a \in 2\pi\mathbb{Z}$) splits sector N into blocks of fixed momentum $k = 2\pi m/L$ (for $L = 14$ and $N = L/2$ of sizes 246, 245, 245, 245, 245, 245, 245, 246, 245, 245, 245, 245, 245, 245); the union of their spectra is the spectrum of the sector to 3.6×10^{-14}, and the ground state of the antiferromagnetic Heisenberg ring has $k = 0$ for even and $k = \pi$ for odd $L/2$ (Marshall's sign rule). Momentum blocks with $k \neq 0, \pi$ are complex in the plane-wave basis even for real Hamiltonians, one of the cases of Section 3 where the field is fixed by the problem, at least in this basis: combined with reflection, time reversal leaves each k-block invariant, so for non-integrable translation-invariant Hamiltonians these blocks show GOE statistics and admit a real basis; the XXZ ring itself is integrable and its blocks are not of GOE type.

- *Beyond dense matrices.* Lanczos with full re-orthogonalisation and restarts, batched over Hamiltonians, reproduces the free-fermion ground-state energies of open XX chains and the Heisenberg ring in sectors up to the sizes of Table 12 (in the full run beyond the reach of dense diagonalisation); Krylov evolution of the domain wall for $L = 22$ (sector dimension 705432) agrees with the single-particle solution to 1.7×10^{-14}.

- *Open chains.* With jump operators S_i^- on every site (Lindblad terms between sectors $N \to N - 1$) and any N-conserving Hamiltonian the particle number performs a pure death process, so the sector distribution must be binomial, $p_n(t) = \binom{N_0}{n} e^{-n\gamma t}(1 - e^{-\gamma t})^{N_0 - n}$; the library reproduces it to 2.4×10^{-10}.

model	sector dim	E_0 (Lanczos)	E_0 (exact)	difference	time [s]
XX open, $L = 16$	12870	-4.9189757237	-4.9189757237	1.8×10^{-15}	0.14
XX open, $L = 20$	184756	-6.1907449998	-6.1907449998	8.9×10^{-16}	0.07
XX open, $L = 24$	2704156	-7.4629855550	-7.4629855550	2.7×10^{-15}	0.56
Heisenberg ring, $L = 16$	12870	-7.1422963606	-7.142296361	$< 5 \times 10^{-10}$	0.059

Table 12: Batched Lanczos on sparse sector Hamiltonians (time includes building the sparse Hamiltonian): ground-state energies against free fermions ($\sum$ of the $L/2$ lowest $\cos \frac{\pi k}{L+1}$) and the tabulated Heisenberg value (nine digits); full run on NVIDIA GeForce RTX 5090.

8 Subalgebras, inclusions and symmetries

The algebras of the previous sections are declared by their sector sizes. Three modules relate algebras to each other and to the symmetries that produce them: `inclusions` treats a subalgebra $\mathcal{N} \subset \mathcal{M}$ with its conditional expectations and indices, `decompose` computes the algebra generated by given matrices or by a finite group, and `su2` uses the full SU(2) symmetry of spin-$\frac{1}{2}$ chains instead of S^z_{tot} alone.

L	dim ($N = L/2$)	E_0 (library)	E_0 (exact)	difference	E_0/L
4	6	-2.000000000	-2.000000000	$< 5 \times 10^{-10}$	-0.50000
6	20	-2.802775638	-2.802775638	$< 5 \times 10^{-10}$	-0.46713
8	70	-3.651093409	-3.651093409	$< 5 \times 10^{-10}$	-0.45639
10	252	-4.515446354	-4.515446354	$< 5 \times 10^{-10}$	-0.45154
12	924	-5.387390917	-5.387390917	$< 5 \times 10^{-10}$	-0.44895
14	3432	-6.263549534	-6.263549534	$< 5 \times 10^{-10}$	-0.44740
16	12870	-7.142296361	-7.142296361	$< 5 \times 10^{-10}$	-0.44639

Table 13: Ground-state energy of the Heisenberg ring $H = \sum_i \mathbf{S}_i \cdot \mathbf{S}_{i+1}$; exact values to nine digits (full run on NVIDIA GeForce RTX 5090).

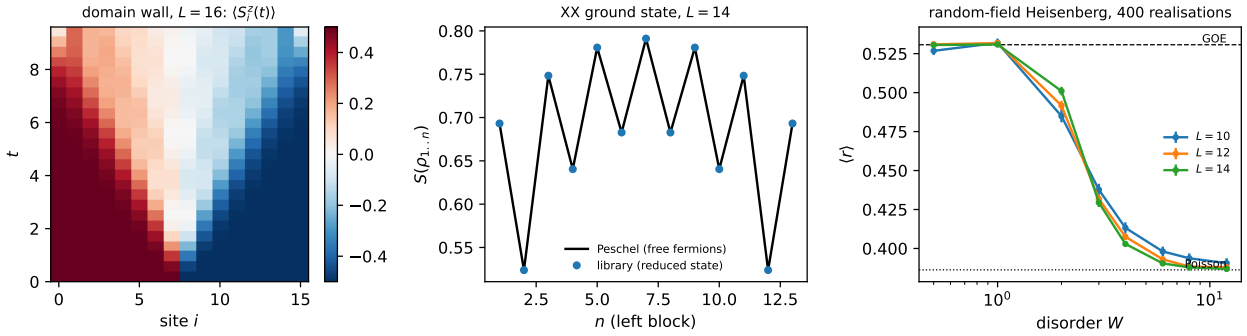


Figure 4: Left: $\langle S_i^z(t)\rangle$ after a domain-wall quench of the XX chain (light cone). Middle: entanglement entropy of the XX ground state, library vs. Peschel's formula. Right: mean level-spacing ratio of the random-field Heisenberg chain, with the GOE and Poisson values.

Inclusions and conditional expectations. Let $\mathcal{N} = \bigoplus_{c=1}^{C} M_{k_c}$ and $\mathcal{M} = \bigoplus_{d=1}^{D} M_{m_d}$. Every unital inclusion $\iota : \mathcal{N} \to \mathcal{M}$ is, up to an inner automorphism $\operatorname{Ad} u$ of $\mathcal{M}$, of the form $\iota(x)_d = \bigoplus_c x_c \otimes 1_{\lambda_{dc}}$, given by the inclusion matrix $\Lambda = (\lambda_{dc}) \in \mathbb{Z}_{\geq 0}^{D\times C}$ with $m_d = \sum_c \lambda_{dc} k_c$ [66]: λ_{dc} is the multiplicity of the summand c of $\mathcal{N}$ in the summand d of $\mathcal{M}$. `Inclusion` stores Λ and an optional (batched) unitary u. With the isometries $V_{dc,i} : \mathbb{C}^{k_c} \to \mathbb{C}^{m_d}$ onto the i-th copy of $\mathbb{C}^{k_c}$, $\iota(x)_d = \sum_{c,i} u_d V_{dc,i} x_c V_{dc,i}^* u_d^*$, so ι is an `InterSectorChannel`, and inclusion matrices of towers multiply. A faithful trace $\tau = \sum_d w_d \operatorname{Tr}_d$ on $\mathcal{M}$ restricts to $\mathcal{N}$ with the weights $\nu = \Lambda^T w$, and

$$E(y)_c = \nu_c^{-1} \sum_{d,i} w_d \, V_{dc,i}^* u_d^* \, y_d \, u_d V_{dc,i}$$

is the unique τ-preserving conditional expectation of $\mathcal{M}$ onto $\mathcal{N}$: it is unital and completely positive, $E \circ \iota = \mathrm{id}$, $E(\iota(n_1) y \, \iota(n_2)) = n_1 E(y) n_2$, and it is the adjoint of ι with respect to τ and $\tau|_{\mathcal{N}}$. The tests check these properties against dense matrices for the three traces of Section 2, for the Markov trace and for random weights. For a connected inclusion (the bipartite graph of Λ is connected) the Perron–Frobenius vector of $\Lambda\Lambda^T$ gives the Markov trace, and the index is $[\mathcal{M} : \mathcal{N}] = \|\Lambda\|^2$ [65, 66]: $(m/k)^2$ for factors $M_k \subset M_m$, n^2 for $\mathbb{C} \subset M_n$ and n for the diagonal subalgebra $\mathbb{C}^n \subset M_n$. The Watatani index $\operatorname{Ind} E = \sum_j u_j u_j^*$ over a quasi-basis $\{u_j\}$ [75] is central, $(\operatorname{Ind} E)_d = (\Lambda\Lambda^T w)_d / w_d$, and equals $[\mathcal{M} : \mathcal{N}]\, 1$ exactly for the Markov trace. The Pimsner–Popa constant $\lambda(E) = \max\{\lambda : E(y) \geq \lambda y \text{ for all } y \geq 0\}$ [67] is $\big(\max_d \sum_c (\nu_c/w_d) \min(k_c, \lambda_{dc})\big)^{-1}$ [67, Theorem 6.1]. For every subfactor of a II_1 factor $\lambda(E) = [\mathcal{M} : \mathcal{N}]^{-1}$ [67, Theorem 2.2], but for Type I inclusions the two differ [67, 6.5]: for $\mathbb{C} \subset M_n$, $\lambda(E) = 1/n$, while the constant of the complete order, $\max\{\lambda :$

$E - \lambda\,\mathrm{id}$ completely positive$\} = 1/\|\operatorname{Ind} E\|$ ([77, Théorème 3.5]; see [76, proof of Proposition 1.5 and Example 1.6]), is $1/n^2$. The tests compare both constants with brute-force values (bisection over rank-one y, and Choi matrices). The basic construction $\mathcal{M}_1 = \langle \mathcal{M}, e_{\mathcal{N}} \rangle$ has the sectors of $\mathcal{N}$, of sizes $(\Lambda^T m)_c$, and $\mathcal{M} \subset \mathcal{M}_1$ has the inclusion matrix Λ^T [65, 66]; the library builds the Jones projection $e_{\mathcal{N}}$ (tested: $e_{\mathcal{N}} y e_{\mathcal{N}} = E(y) e_{\mathcal{N}}$, $e_{\mathcal{N}} \in \mathcal{N}'$, the Markov property $\tau_1(y e_{\mathcal{N}}) = [\mathcal{M} : \mathcal{N}]^{-1} \tau(y)$), the Jones tower $\mathcal{N} \subset \mathcal{M} \subset \mathcal{M}_1 \subset \dots$ with constant index, and the relative commutant $\mathcal{N}' \cap \mathcal{M} = \bigoplus_{\lambda_{dc} > 0} M_{\lambda_{dc}}$. All maps act on batches by index gathers, without host synchronisation, and are differentiable.

The spin-chain tower. The $U(1)$ algebras $A_L = \bigoplus_{N=0}^{L} M_{\binom{L}{N}}$ of Section 7 form the tower $A_L \subset A_{L+1}$, $x \mapsto x \otimes 1_2$, whose Bratteli diagram is Pascal's triangle: sector N of A_{L+1} receives the sectors $N-1$ (new spin up) and N (new spin down) of A_L. Λ is the incidence matrix of the path with $2L+3$ vertices, so

$$[A_{L+1} : A_L] = \|\Lambda\|^2 = 4\cos^2 \frac{\pi}{2L+4} \;\nearrow\; 4,$$

values $4\cos^2(\pi/n)$, $n = 4, 6, 8, \dots$ $(2, 3, 2+\sqrt{2}, \dots)$, of the discrete series below 4 in Jones' index theorem [65]. The tests check the spectrum of the path graph and the index for $L \le 8$, that the embedding is $x \otimes 1_2$ in the basis of `SpinChain`, and that the conditional expectation for the normalised trace of $(\mathbb{C}^2)^{\otimes(L+1)}$ is the normalised partial trace over the new site. The program `listings/inclusions.py` shows the interface: for $M_2 \oplus \mathbb{C} \subset M_4 \oplus M_3$ with a random unitary it computes the index, the Markov conditional expectation and its bimodule property, the Watatani index, both Pimsner–Popa constants and the Jones projection of the basic construction, and the indices of the spin-chain tower (`inclusions.Inclusion`, `index`, `conditional_expectation`, `pimsner_popa_constant`, `basic_construction`, `jones_projection`, `chain_tower`).

From a symmetry to the algebra: Wedderburn decomposition. Section 1 obtained the algebra (1) from a symmetry through the commutant. `decompose` carries out this step numerically. Given matrices $S_1, \dots, S_J \in M_n(\mathbb{C})$ (for instance the generators of a Lie algebra representation) or all elements $\pi(g)$ of a unitary representation of a finite group, it returns a unitary V with

$$V^* \mathcal{A} V = \bigoplus_c M_{k_c}(\mathbb{C}) \otimes 1_{m_c}, \qquad V^* \mathcal{A}' V = \bigoplus_c 1_{k_c} \otimes M_{m_c}(\mathbb{C}),$$

for the von Neumann algebra $\mathcal{A} = \{S_j, S_j^*\}''$ generated by the matrices, together with maps between $n \times n$ matrices and operators of the algebras $\bigoplus_c M_{k_c}$ and $\bigoplus_c M_{m_c}$ of the library. The algorithm combines two randomised algorithms of Murota and coauthors. As in Maehara and Murota [80], who apply the method of Murota, Kanno, Kojima and Kojima [78] to the commutant, the eigenspaces of a random Hermitian element of $\mathcal{A}'$ are, generically, irreducible $\mathcal{A}$-submodules, one per copy of each irreducible representation; the copies are then grouped and aligned as in [78, Sec. 4.2], here with elements of $\mathcal{A}'$ instead of $\mathcal{A}$. For a second random element $Y \in \mathcal{A}'$ and orthonormal bases Q_i, Q_j of two such submodules, $Q_i^* Y Q_j$ intertwines them; by Schur's lemma it vanishes if they are inequivalent and is a non-zero multiple of a unitary (generically) if they are equivalent, so the non-zero blocks give the equivalence classes and their polar factors align the bases of the copies. Random elements of $\mathcal{A}'$ come from the twirl $|G|^{-1} \sum_g \pi(g) X \pi(g)^*$ for a finite group, or from the null space of $X \mapsto ([H_j, X])_j$ with the Hermitian matrices H_j given by $(S_j + S_j^*)/2$ and $(S_j - S_j^*)/2i$ (as in [80]), whose size is reduced by first diagonalising a random Hermitian element of $\mathcal{A}$ (the first step of [78]). Every result is verified before it is returned: the generators lie in $\mathcal{B} = V\big(\bigoplus_c M_{k_c} \otimes 1_{m_c}\big)V^*$, fresh random elements of $\mathcal{A}'$ lie in $\mathcal{B}'$, and $\sum_c m_c^2 = \dim \mathcal{A}'$, which together give $\mathcal{A} = \mathcal{B}$ by the double commutant theorem; a failed check triggers a new random draw. The decomposition is over $\mathbb{C}$ and uses dense linear algebra. For an algebra closed under complex conjugation (real generators) entrywise conjugation

maps every sector onto a sector, $\overline{V_c} = V_d(J_c \otimes M_c)$ by Schur's lemma, and `real_structure` reads off the type of every simple summand of the real algebra (cf. [79] for the numerical decomposition of real matrix $*$-algebras with general irreducible components): complex for $d \neq c$ (a pair of conjugate sectors), real for $d = c$ and $J_c\overline{J_c} = 1$, quaternionic for $J_c\overline{J_c} = -1$; the sign has a gap of 2, so the decision needs no tolerance, and `real_basis` returns a real orthogonal basis adapted to the types (Takagi factorisation of the symmetric unitary J_c). In a quaternionic sector the multiplicity block of a real symmetric commuting operator has doubly degenerate eigenvalues (Kramers). The tests recover block structures hidden by random unitaries, the Fourier basis of the regular representation of $\mathbb{Z}_n$, and Schur–Weyl duality [81]: on $(\mathbb{C}^2)^{\otimes L}$, $L \leq 6$, the algebra generated by the permutations of the tensor factors and the algebra generated by the total spin have the same central projections, with k_c and m_c exchanged (the dimensions of the irreducible representations of S_L of shape $(L-j, j)$ and $2S+1$, $S = \frac{L}{2} - j$). The program `listings/symmetry.py` shows this for $L = 4$ (`decompose.from_group` with `permutation_representation`, `decompose.from_generators` with `total_spin`) and computes the SU(2) blocks and the lowest energy per total spin of a Heisenberg ring of 14 spins (`SU2Chain`, `heisenberg`, `ground_state`, `lowest_energies`).

SU(2) symmetry of spin chains. The Heisenberg Hamiltonian $H = \sum J_{ij}\, \mathbf{S}_i \cdot \mathbf{S}_j$ commutes with all of SU(2), not only with S^z_{tot}. On $(\mathbb{C}^2)^{\otimes L} = \bigoplus_S \mathbb{C}^{m_S} \otimes V_S$, with V_S the spin-S representation,

$$\mathrm{SU}(2)' = \bigoplus_S M_{m_S}(\mathbb{C}), \qquad m_S = \binom{L}{L/2 - S} - \binom{L}{L/2 - S - 1}, \qquad \sum_S (2S+1)\, m_S = 2^L,$$

and an invariant Hamiltonian is $\bigoplus_S H_S \otimes 1_{2S+1}$: every multiplet is computed once instead of $2S+1$ times, and the blocks shrink from $\binom{L}{L/2+S}$ to m_S. `SU2Chain` realises $\mathbb{C}^{m_S}$ as the highest-weight vectors $\ker S^+ \cap \{S^z_{\mathrm{tot}} = S\}$ in the $U(1)$ sector $N = L/2 + S$ of `SpinChain`. Their orthonormal basis B_S is built site by site with Clebsch–Gordan coefficients (the Yamanouchi basis, labelled by the sequence of intermediate spins $S_1 = \frac{1}{2}, \ldots, S_L = S$); it is orthonormal by construction, so no null space or orthogonalisation is needed, and it is expressed directly in the $U(1)$ basis, where the block of an invariant operator is $A_S = B_S^* A_N B_S$. Clebsch–Gordan coefficients are computed exactly in rational arithmetic (Racah's formula [82], Condon–Shortley phases). By the Wigner–Eckart theorem [83] the reduced matrix elements of the site spins follow from highest-weight vectors alone, and $\mathbf{S}_i \cdot \mathbf{S}_j$ is assembled from them as a sum over intermediate spins, so spin correlations need no sum over magnetic quantum numbers. For $L = 14$ the largest $U(1)$ block, $\binom{14}{7} = 3432$, becomes $m_1 = m_2 = 1001$, and the cubic cost of diagonalising all blocks, $\sum_N \binom{L}{N}^3$ against $\sum_S m_S^3$, drops by a factor of about 48; for $L = 16$ the largest block shrinks from 12870 to 3640. The bases of all spins hold $\sum_S m_S \binom{L}{L/2+S}$ numbers ($1.0 \cdot 10^8$ for $L = 16$), which limits dense SU(2) computations to about $L = 16$. The tests compare the full spectra of open and periodic chains, with a field and next-nearest-neighbour couplings, for $L \leq 12$ with dense diagonalisation (without next-nearest-neighbour couplings also with the $U(1)$ sectors; to 10^{-10}), and reproduce the ring energies of Table 13 for $L \leq 14$ (to 10^{-9}, with total spin 0), the theorem of Lieb and Mattis [84] for open chains and even rings with $L \leq 12$ (the ground state has $S = 0$ for even and $S = \frac{1}{2}$ for odd L, and the lowest energy $E_0(S)$ increases strictly with S), the two exactly degenerate singlets with $E = -3L/8$ at the Majumdar–Ghosh point $J_2 = J/2$ [85, 86], textbook and SymPy values of the Clebsch–Gordan coefficients, and Wigner–Eckart matrix elements and spin correlations against direct evaluation.

Section 10 uses the conditional expectations of this section to compute the Connes–Størmer relative entropy of two maximal abelian subalgebras of M_n.

9 Use case: noisy circuits with a conserved charge

Quantum algorithms that preserve a charge offer a cheap test for errors: if the ideal evolution keeps the charge, a measurement of the charge at the end detects every error that changed it, and discarding those runs (*symmetry verification* [36, 37]) improves the accepted results. The XY mixers of the quantum alternating operator ansatz keep the Hamming weight [38]. In the language of this library the register then carries the algebra $\mathcal{M} = \bigoplus_{N=0}^{n} M_{\binom{n}{N}}$, the measurement of the weight is a central projection P_w, and post-selection is the Lüders conditioning on it. Noise is a Lindblad generator whose jumps either stay in a sector or move between sectors; so the noisy density matrix never leaves $\mathcal{M}$, and the whole question is a computation in the algebra.

Model. A ring of n qubits evolves under the XY mixer $H = \frac{1}{2}\sum_{\langle ij\rangle}(S_i^+S_j^- + S_i^-S_j^+) = \frac{1}{4}\sum_{\langle ij\rangle}(X_iX_j + Y_iY_j)$ for a time $T = 2$ from $|1^w0^{n-w}\rangle$, $w = n/2$, with three kinds of local noise: amplitude damping S_i^- (rate $\gamma_\downarrow$, from sector N to $N-1$), thermal excitation S_i^+ ($\gamma_\uparrow$, to $N+1$) and dephasing S_i^z (γ_φ, inside the sectors). With ψ the noiseless state at time T we compute the acceptance probability $p = \mathrm{Tr}(P_w\rho(T))$, the fidelity $F = \langle\psi|\rho(T)|\psi\rangle$ and the fidelity after post-selection $F_{\mathrm{post}} = F/p$.

Proposition 1. *Let $I = \int_0^T \sum_i \big(\frac{1}{4} - \langle S_i^z\rangle^2_{\psi(t)}\big)\,dt$ along the noiseless trajectory.*

(a) If $\gamma_\uparrow = \gamma_\varphi = 0$: $p = e^{-w\gamma_\downarrow T}$ and $F_{\mathrm{post}} = 1$ for every $\gamma_\downarrow$, n and T.

(b) If $\gamma_\varphi = 0$: $1 - F_{\mathrm{post}} = O(\gamma_\downarrow\gamma_\uparrow T^2)$.

(c) If $\gamma_\downarrow = \gamma_\uparrow = 0$: $p = 1$ and $1 - F_{\mathrm{post}} = 1 - F = \gamma_\varphi I + O(\gamma_\varphi^2)$.

Proof. Since $\sum_i S_i^+S_i^- = N$ and $\sum_i S_i^-S_i^+ = n - N$ are central, the no-jump part of the generator is $\rho \mapsto -i(H_{\mathrm{eff}}\rho - \rho H_{\mathrm{eff}}^*)$ with $H_{\mathrm{eff}} = H - \frac{i}{2}(\gamma_\downarrow N + \gamma_\uparrow(n-N))$, and on sector w it is the Hamiltonian evolution damped by the factor $e^{-(\gamma_\downarrow w + \gamma_\uparrow(n-w))t}$. In the Dyson expansion in the jumps, the part of $\rho(T)$ in sector w consists of this no-jump term, proportional to $|\psi\rangle\langle\psi|$, and of the terms with equally many up and down jumps. (a) Without up jumps the second kind is absent, so $P_w\rho(T)P_w = e^{-w\gamma_\downarrow T}|\psi\rangle\langle\psi|$. (b) The terms with $m \geq 1$ up and m down jumps have total weight $O((\gamma_\downarrow\gamma_\uparrow T^2)^m)$ against a no-jump weight of order one. (c) Dephasing keeps the sector, and the first-order Dyson term gives $1 - F = \gamma_\varphi \int_0^T \sum_i (\langle(S_i^z)^2\rangle - \langle S_i^z\rangle^2)\,dt$ with $(S_i^z)^2 = \frac{1}{4}$. □

Statement (a) is the reason why symmetry verification removes amplitude damping completely at zero temperature: the damping commutes with the post-selection. Statements (b) and (c) say what remains: thermal noise is suppressed from first to second order, dephasing is not detected at all.

Computation. All noise models and rates are one batch: 28 settings (four noise models, seven rates between 10^{-3} and 10^{-1}) per n, with the rates folded into the coefficients of the jumps (`SpinChain.sector_jump(i, kind).scaled(...)`), one call of `lindblad_evolve` with the completely positive splitting (`method='kraus'`, 40 steps), the central projection `algebra.central` and `condition_on`. The jumps are `SectorJumps` (Section 3); as dense maps between sectors they would need $3n$ tensors of $(n+1)^2\binom{n}{n/2}^2$ entries, 2.3 GB each for $n = 12$. Table 14 and Figure 5 show the results for $n = 4, \ldots, 10$.

Results. The computation confirms Proposition 1 in every case: at zero temperature the post-selected state agrees with the noiseless one to $1e-15$ and the acceptance with $e^{-w\gamma T}$ to $6e-6$ (the error of the time steps); the thermal infidelity after post-selection has slope 2.00; the dephasing infidelity agrees with its first-order value to 0.1%. The integral I grows linearly with n (each qubit

					(a) T_1, zero temp.		(b) thermal	(c) dephasing		mixed, $\gamma = 0.01$	
n	$\binom{n}{w}$	$\sum_N \binom{n}{N}^2$	4^n	time [s]	err. p	$\lvert 1 - F_{\text{post}} \rvert$	slope	I	$\frac{1-F_{\text{post}}}{\gamma_\varphi I}$	$1 - F$	$1 - F_{\text{post}}$
4	6	70	256	0	4e-7	9e-16	2.00	1.167	0.999	0.1126	0.0254
6	20	924	4096	1	6e-6	1e-15	2.00	1.380	0.999	0.1573	0.0307

Table 14: Noisy XY mixer on a ring of n qubits, $T = 2$ (CPU, double precision). Dimensions: the weight sector, the algebra and a dense density matrix. (a) Relative deviation of p from $e^{-w\gamma T}$ (the error of the second-order splitting) and $|1 - F_{\text{post}}|$, maxima over the rates. (b) Slope of $\log(1 - F_{\text{post}})$ against $\log \gamma$ for $\gamma_\uparrow = 0.2\gamma_\downarrow = 0.2\gamma$, at small γ. (c) The integral I of Proposition 1 and the ratio of the computed infidelity to its first-order value at $\gamma_\varphi = 10^{-3}$. Mixed: $\gamma_\downarrow = \gamma_\varphi = \gamma$, $\gamma_\uparrow = 0.1\gamma$, without and with post-selection.

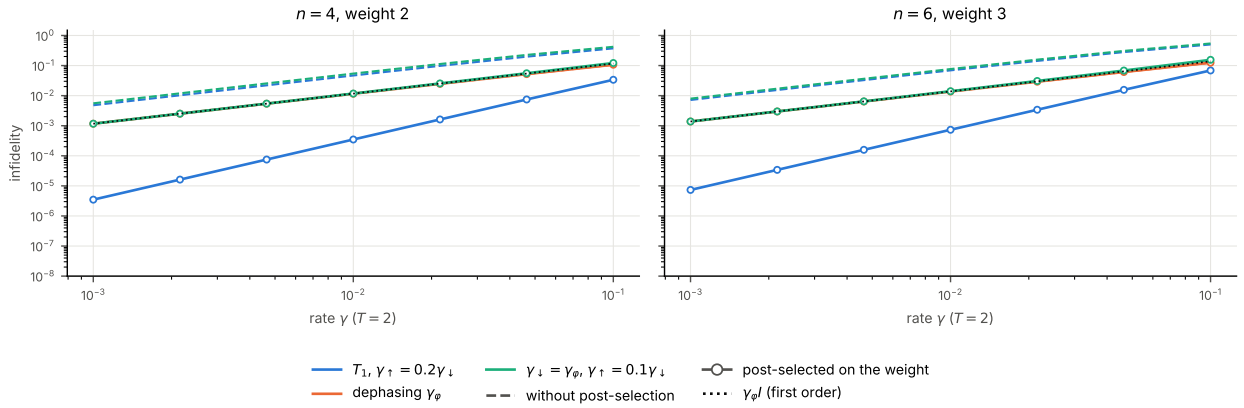


Figure 5: Infidelity of the noisy XY mixer without (dashed) and with (solid) post-selection on the Hamming weight, one panel per n. Thermal T_1 noise is suppressed from first to second order (slope 2), dephasing passes undetected and follows its first-order value $\gamma_\varphi I$ (dotted), and for T_1 at zero temperature (not shown) the post-selected state is exact. For realistic mixed noise post-selection removes the T_1 part, and dephasing remains.

that leaves its initial value contributes), so the undetected dephasing error grows with the register while the detected part does not reach the accepted runs. For the mixed model, at a rate of 0.01 per unit of time, post-selection reduces the infidelity by a factor of five to six for every n (Table 14), and what is left is dephasing. For algorithm design this quantifies the value of the symmetry: it protects against energy relaxation, not against dephasing, and the remaining error is predicted to first order by a single integral over the noiseless trajectory.

10 Applications in the companion papers

The library was built for, and used in, four further studies, which are reported with proofs, validation and figures in companion papers; here we only name the parts of the library they use.

How strongly trace characterisations fail (with S. A. Abed [1]). For 25 inequalities that characterise the trace, the violation by $\varphi_W = \mathrm{Tr}(W\,\cdot\,)$ is maximised over all operators at every contrast $\Delta(W)$, in one batched optimisation per inequality (`optimize.SpectrumParam`, `PositiveParam`, `UnitaryParam`; lattice operations $P \wedge Q$, $P \vee Q$ by functional calculus with Daleckii–Krein gradients), and the best points are recomputed in mpmath with 40 digits. The computed envelopes led to closed

forms, proved in [1], that vanish at the trace to the orders δ, δ^2, δ^4 and δ^8.

The Connes–Størmer entropy of two MASAs [2]. The relative entropy $H(\Delta_n|U\Delta_n U^*)$ of the diagonal algebra and a rotated copy is a supremum over partitions of unity; with the conditional expectations of `Inclusion` and partitions parametrised by isometries, many unitaries and starts form one batch. For $n = 2$ the computation suggested a closed form, proved in [2]; for $n = 3, 4, 5$ it finds partitions that beat every orthonormal basis found.

A boundary time crystal under local dissipation [3]. Local emission and dephasing of N two-level systems commute with the permutations of the sites, so the state stays in the Schur–Weyl algebra $\bigoplus_j M_{2j+1}\otimes 1_{m_j}$, and the local dissipators are maps between its sectors (`PermutationalLindbladian`). With all sectors kept, the lifetimes of the oscillations follow for $N \leq 80$ without truncation, ten rates in one batch.

Symmetry-resolved entanglement in the XXZ chain [3]. The reduced state of a charge-conserving state is an operator of $\bigoplus_q M_{d_q}$, so resolved entropies and charge statistics are read off its blocks. For $L = 24$ one batched Lanczos run gives the ground states for eight anisotropies, and the charge statistics give the Luttinger parameter to about one per cent.

11 Comparison with NumPy/SciPy

To separate what the library adds from what plain array code already gives, the same computations were written three ways (`scripts/baselines.py`; Table 15). (A) The Monte Carlo of Exp. 1: B pairs of positive operators with prescribed contrast and Haar-orthogonal eigenvectors in every sector, a Haar-orthogonal $U \in \mathcal{M}$, and z. The direct NumPy/SciPy formulation loops over samples and sectors with `scipy.stats.ortho_group`; a hand-vectorised NumPy version uses stacked QR decompositions with the phase correction; the library needs three calls. (B) Ground states of sector Hamiltonians: `scipy.sparse.linalg.eigsh` (ARPACK [45]), one call per Hamiltonian, against the batched Lanczos solver. The three Monte Carlo versions sample the same distribution: their means differ by at most 1.2 standard errors, a two-sample Kolmogorov–Smirnov test of the library against vectorised NumPy gives $p \geq --$, and the ground-state energies agree to round-off. All times are medians of up to five runs after a warm-up.

The comparison is instructive in both directions. On a CPU the gain over the loop depends on the block size: it is large for small blocks, where the Python overhead of the loop dominates, and small for large blocks, where both spend their time in the same LAPACK routines. Against carefully vectorised NumPy the library is not systematically faster on a CPU (Table 15): both call the same kernels. Its value there is that the vectorised, correctly sampled, block-aware version is the default rather than an expert rewrite, and that the same code runs on a GPU and can be differentiated. The NumPy versions work in double precision, so the library's CPU and GPU columns do too; the last column repeats the GPU run in single precision, since double precision is 32–64 times slower on consumer GPUs. For ground states, ARPACK is the reference for a single Hamiltonian on a CPU; our Lanczos solver re-orthogonalises fully for robustness and is batched over Hamiltonians, which is what lets disorder averages and GPUs pay off (Table 15).

C	k	NumPy loop	NumPy vectorised	library (CPU)	library (GPU)	GPU, FP32
1	2	160	2.49	1.50	124	
16	16	2852	690	884	2000	
32	16	5702	1380	1688	4005	

L	sector dim	Hamiltonians	`eigsh` [s]	library, CPU [s]	library, GPU [s]	$\max \lvert\Delta E_0\rvert$
16	12870	1	0.0288	0.0552	6.2e-15	
16	12870	20	0.621	0.114	4.3e-14	
18	48620	1	0.115	0.0588	3.6e-15	
20	184756	1	0.540	0.0642	1.1e-14	

Table 15: Top: time per Monte Carlo sample [μs] for (A); bottom: ground states of Heisenberg rings (one Hamiltonian) and random-field chains ($W = 3$, several Hamiltonians) in the sector $N = L/2$, total time for all Hamiltonians ("–": no GPU in this run) (full run on NVIDIA GeForce RTX 5090).

12 Performance

Table 16 and Figure 6 report the wall-clock time per sample of library operations (not of isolated kernels), in single precision, with batch $B = \min\bigl(4096,\, 2^{22}/(Ck^2)\bigr)$ (shown in the table) (full run on NVIDIA GeForce RTX 5090; CPU with 16 threads; PyTorch 2.14.1+cu130, CUDA 13.0, cuSOLVER 12004). The versions matter: batched Hermitian eigensolvers on GPUs cover all block sizes only in recent cuSOLVER and PyTorch releases (older releases fall back to a loop over the matrices for blocks larger than 32), and batched SVD on GPUs is limited to blocks of size at most 32, which is why the library uses the Hermitian eigensolver wherever the operator is self-adjoint (and the eigensolver of the Hermitian dilation $\left(\begin{smallmatrix} 0 & A \\ A^* & 0 \end{smallmatrix}\right)$ for singular values of larger blocks). Batched QR on GPUs factorises small matrices one at a time; Haar sampling therefore uses a batched Gram–Schmidt orthonormalisation with reorthogonalisation on GPUs, which gives the same matrices as QR with the phase correction. On the CPU, PyTorch's batched eigen-, singular-value and QR decompositions loop over the matrices on one core; the library splits such batches into one chunk per thread (identical results), so the CPU column uses all cores. The operations are: generating random operators with prescribed spectra, $f(A)$ by eigendecomposition, $|A|$ (singular values), the Michelson contrast, applying a channel, the Lindblad channel $e^{t\mathcal{L}}$, and one gradient step of the optimisation; the table gives the CPU and GPU times and their ratio.

Besides speed, the structure of the computation is checked directly (`scripts/gpu_check.py` on a GPU, `tests/test_sync_free.py` on a CPU): the operations read no values back to the host (each such read synchronises a GPU), and their number of kernel launches does not grow with the number of sectors C at a fixed total size. Per-sector work of this kind made the random generation of an earlier version of the library slower on a GPU than on a CPU; the check would have found it.

The first version of this work reported GPU speedups for raw kernels on a P100 and found them only for part of the configurations. That observation reflects where GPUs help: a GPU needs enough independent work per kernel launch. For computations in (1) this means many samples, many sectors or large blocks, and matrix products rather than decompositions of small matrices. The design follows from it: everything is batched, so that the batch dimension supplies the parallel work, and the cost module warns before configurations that would be slow. For workloads that are small or dominated by a single eigenproblem, the CPU (or SciPy, Section 11) is the right tool, and the library runs there unchanged.

operation	k	C	batch	GPU [μs]	CPU [μs]	speedup
generate	4	1	4096	63.5	1.90	0.0299
f(A) eigh	4	1	4096	0.160	3.06	19.2
|A| SVD	4	1	4096	0.388	5.27	13.6
contrast	4	1	4096	0.0979	1.44	14.7
channel	4	1	4096	0.116	0.186	1.60
exp(tL)	4	1	4096	2.05	34.1	16.6
opt. step	4	1	4096	2.76	7.78	2.82
generate	4	16	4096	995	28.1	0.0283
f(A) eigh	4	16	4096	0.514	44.7	86.9
|A| SVD	4	16	4096	4.17	80.7	19.4
contrast	4	16	4096	0.358	22.0	61.5
channel	4	16	4096	1.34	4.53	3.38
exp(tL)	4	16	1024	49.2	556	11.3
opt. step	4	16	4096	9.54	80.2	8.41
generate	4	256	1024	15710	534	0.0340
f(A) eigh	4	256	1024	9.01	742	82.3
|A| SVD	4	256	1024	66.3	1283	19.4
contrast	4	256	1024	5.63	359	63.7
channel	4	256	1024	20.9	65.0	3.12
exp(tL)	4	256	64	7743	15365	1.98
opt. step	4	256	1024	133	1604	12.0
generate	16	1	4096	60.9	17.3	0.283
f(A) eigh	16	1	4096	0.206	23.8	116
|A| SVD	16	1	4096	0.780	44.5	57.0
contrast	16	1	4096	0.137	10.9	79.5
channel	16	1	4096	0.129	1.72	13.4
exp(tL)	16	1	64	1149	13608	11.8
opt. step	16	1	4096	3.61	74.3	20.6
generate	16	16	1024	1032	290	0.281
f(A) eigh	16	16	1024	1.76	419	238
|A| SVD	16	16	1024	10.7	746	69.6
contrast	16	16	1024	1.26	182	145
channel	16	16	1024	1.63	65.0	39.8
exp(tL)	16	16	4	79221	181417	2.29
opt. step	16	16	1024	34.9	1505	43.1
generate	16	256	64	18838	5937	0.315
f(A) eigh	16	256	64	78.0	6736	86.3
|A| SVD	16	256	64	172	11860	68.9
contrast	16	256	64	36.9	2871	77.7
channel	16	256	64	26.3	989	37.6
exp(tL)	16	256	1	2349043	3558629	1.51
opt. step	16	256	64	567	21657	38.2
generate	64	1	1024	62.0	329	5.30
f(A) eigh	64	1	1024	3.47	393	113
|A| SVD	64	1	1024	13.1	820	62.4
contrast	64	1	1024	2.48	211	85.0
channel	64	1	1024	0.859	71.2	82.9
opt. step	64	1	1024	34.2	1737	50.8
generate	64	16	64	1262	4995	3.96
f(A) eigh	64	16	64	59.2	6495	110
|A| SVD	64	16	64	211	12966	61.4
contrast	64	16	64	41.2	3424	83.0
channel	64	16	64	13.7	1054	76.8
opt. step	64	16	64	550	28154	51.2
generate	64	256	4	132631	103301	0.779
f(A) eigh	64	256	4	1753	99206	56.6
|A| SVD	64	256	4	3398	204740	60.2
contrast	64	256	4	933	54526	58.4
channel	64	256	4	217	12346	57.0
opt. step	64	256	4	8867	448973	50.6

Table 16: Wall-clock time per sample of library operations for C sectors of size k and the batch size shown (full run on NVIDIA GeForce RTX 5090, CPU with 16 threads) (single precision; "exp(tL)" uses a smaller batch because its superoperators have k^4 entries). With a GPU the table also gives the CPU time and the speedup CPU/GPU.

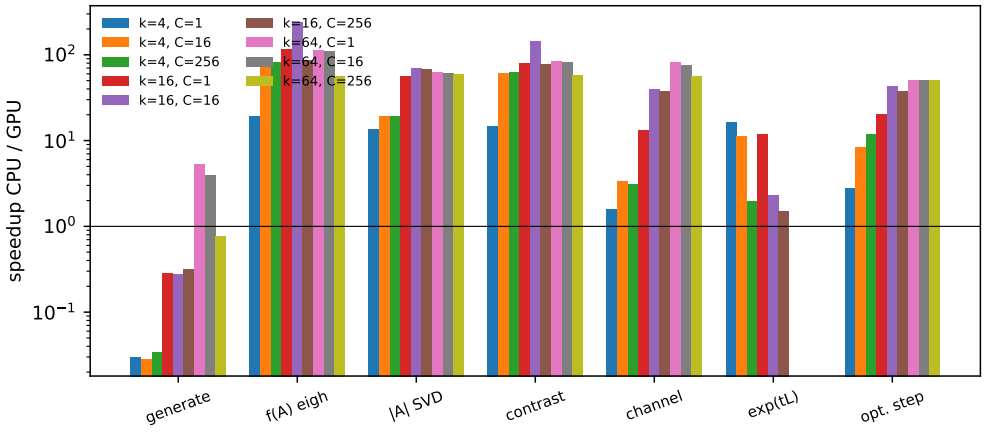


Figure 6: Speedup CPU/GPU (logarithmic axis; above 1 the GPU is faster) of the operations of Table 16, one bar per configuration (k, C).

13 Limitations and roadmap

- *Layout.* Blocks are padded to $k_{\max}$. For very different sector sizes (binomial sectors of spin chains) one works in single sectors, and `SU2Chain` builds the block of a single total spin without padding; operators acting on all sectors at once would need a bucketed layout, which is not implemented yet. Matrix products of padded operators also multiply the padding: for the ten-qubit register of Section 9 the dense Hamiltonian part of a time step costs about four times $\sum_N k_N^3$, and it, not the jumps, dominates the run time.

- *Symmetries.* Abelian charges with an arbitrary fusion rule and lattice translations are supported, and for spin-$\frac{1}{2}$ chains the SU(2) multiplets reduce the blocks further (`su2`; dense up to $L \approx 16$, Section 8). Not yet implemented: SU(2) combined with momentum sectors, chains of spins larger than $\frac{1}{2}$, reflections and spin flip, and sparse (Lanczos) solvers in the SU(2) blocks. The numerical Wedderburn decomposition handles any symmetry given by matrices, but with dense linear algebra; it identifies the real, complex and quaternionic summands of real algebras, but the core algebra class stores only real and complex blocks (quaternionic blocks are kept as complex blocks together with their J_c). Dense methods reach about 16–18 spins in the $U(1)$ sectors, the sparse Lanczos and Krylov solvers handle the half-filled sector of 24 spins (dimension $2.7 \cdot 10^6$; Table 12 in the full run), and about 26 spins on a GPU with tens of gigabytes (an estimate from the memory of the Krylov basis); tensor-network methods are out of scope.

- *Optimisation.* The search is local; envelopes are lower bounds for suprema (upper bounds for infima). Gradients of the functional calculus are finite at degenerate spectra, but gradients through the eigenvectors returned by `eigh` are not; second derivatives of $f(A)$ are correct at non-degenerate spectra only.

- *Divergences.* For $\alpha < 1$ the sandwiched divergence depends on all eigenvalues of $\sigma^g \rho \sigma^g$, $g = (1-\alpha)/(2\alpha)$, whose condition number grows without bound as $\alpha \to 0$; for small α and ill-conditioned states the computed value is not accurate (hence the restriction $\alpha \geq 0.2$, $\rho, \sigma \geq \varepsilon/d$ in Section 6.1).

- *Scale.* Choi matrices, superoperators and $e^{t\mathcal{L}}$ work with $k_c^2 \times k_c^2$ matrices and are limited to blocks of a few tens; RK4 integration scales to larger blocks.

- *Scope.* Finite dimensions and Type I only; no tensor-network methods, and sparse methods only for sector Hamiltonians (Lanczos and Krylov) and for jumps that map basis vectors to basis vectors (`SectorJump`); other sparse jumps between sectors are stored densely.

14 Availability and reproducibility

The code is available under the MIT licence at

`https://github.com/andre-a-no/von_neumann_type_i`.

`bash scripts/run_all.sh full cuda` regenerates every table and figure of this text on a GPU; `bash scripts/run_all.sh` runs a reduced version on a CPU in a few minutes (written to `.quick_output/`, not into the paper's tables), and `python scripts/gpu_check.py` checks a GPU in a few minutes (correctness against the CPU, absence of per-sector work, timings). The tables state the device and the mode they were produced with; the JSON file next to each table also records the commit of the code.

References

[1] S. A. Abed, I. A. Nikolaeva, A. A. Novikov, "Trace-characterising inequalities fail with the Michelson contrast of the functional and of the operators, and projections detect the failure", preprint (2026); source in `man/trace_envelopes` of the repository.

[2] I. Nikolaeva, A. Novikov, "The Connes–Størmer entropy of two maximal abelian subalgebras: a closed form for M_2 and partitions that beat bases in M_n", preprint (2026); source in `man/connes_stormer` of the repository.

[3] I. Nikolaeva, A. Novikov, "Exact symmetry sectors at work: finite-N lifetimes of a boundary time crystal under local dissipation and symmetry-resolved entanglement in the XXZ chain", preprint (2026); source in `man/btc_sre` of the repository.

[4] I. Nikolaeva, A. Novikov, "Finite-dimensional Type I von Neumann algebras in PyTorch: a GPU-accelerated framework for random block-diagonal operators", arXiv:2606.15882v1 (2026).

[5] L. T. Gardner, "An inequality characterizes the trace", Canad. J. Math. 31 (1979), no. 6, 1322–1328.

[6] D. Petz, J. Zemánek, "Characterizations of the trace", Linear Algebra Appl. 111 (1988), 43–52.

[7] A. A. Novikov, O. E. Tikhonov, "Characterization of central elements of operator algebras by inequalities", Lobachevskii J. Math. 36 (2015), no. 2, 208–210; arXiv:1502.01267.

[8] D. Virosztek, "Connections between centrality and local monotonicity of certain functions on C*-algebras", J. Math. Anal. Appl. 453 (2017), 221–226.

[9] S. A. Abed, I. A. Nikolaeva, A. A. Novikov, "Generalisation of Michelson contrast for operators and its properties", Lobachevskii J. Math. 45 (2024), no. 8, 3835–3848; arXiv:2010.10130.

[10] J. R. Johansson, P. D. Nation, F. Nori, "QuTiP 2: A Python framework for the dynamics of open quantum systems", Comput. Phys. Commun. 184 (2013), 1234–1240.

[11] N. Lambert et al., "QuTiP 5: The Quantum Toolbox in Python", Phys. Rep. 1153 (2026), 1–62; arXiv:2412.04705.

[12] A. Javadi-Abhari et al., "Quantum computing with Qiskit", arXiv:2405.08810 (2024).

[13] P. Guilmin, A. Bocquet, É. Genois, D. Weiss, R. Gautier, "Dynamiqs: an open-source Python library for GPU-accelerated and differentiable simulation of quantum systems", in preparation (2025), `https://github.com/dynamiqs/dynamiqs`.

[14] F. Mezzadri, "How to generate random matrices from the classical compact groups", Notices Amer. Math. Soc. 54 (2007), 592–604.

[15] M. L. Mehta, *Random Matrices*, 3rd ed., Elsevier, 2004.

[16] D. N. Page, "Average entropy of a subsystem", Phys. Rev. Lett. 71 (1993), 1291–1294.

[17] S. K. Foong, S. Kanno, "Proof of Page's conjecture on the average entropy of a subsystem", Phys. Rev. Lett. 72 (1994), 1148.

[18] J. Sánchez-Ruiz, "Simple proof of Page's conjecture on the average entropy of a subsystem", Phys. Rev. E 52 (1995), 5653–5655.

[19] E. Lubkin, J. Math. Phys. 19 (1978), 1028.

[20] B. Collins, P. Śniady, "Integration with respect to the Haar measure on unitary, orthogonal and symplectic group", Commun. Math. Phys. 264 (2006), 773–795.

[21] K. Życzkowski, H.-J. Sommers, "Induced measures in the space of mixed quantum states", J. Phys. A 34 (2001), 7111–7125.

[22] I. Peschel, "Calculation of reduced density matrices from correlation functions", J. Phys. A 36 (2003), L205–L208.

[23] V. Oganesyan, D. A. Huse, "Localization of interacting fermions at high temperature", Phys. Rev. B 75 (2007), 155111.

[24] Y. Y. Atas, E. Bogomolny, O. Giraud, G. Roux, "Distribution of the ratio of consecutive level spacings in random matrix ensembles", Phys. Rev. Lett. 110 (2013), 084101.

[25] M. Takesaki, *Theory of Operator Algebras I*, Springer, 2002.

[26] G. C. Wick, A. S. Wightman, E. P. Wigner, "The intrinsic parity of elementary particles", Phys. Rev. 88 (1952), 101–105.

[27] S. D. Bartlett, T. Rudolph, R. W. Spekkens, "Reference frames, superselection rules, and quantum information", Rev. Mod. Phys. 79 (2007), 555–609.

[28] W. H. Zurek, "Decoherence, einselection, and the quantum origins of the classical", Rev. Mod. Phys. 75 (2003), 715–775.

[29] A. W. Sandvik, "Computational studies of quantum spin systems", AIP Conf. Proc. 1297 (2010), 135–338.

[30] P. Weinberg, M. Bukov, "QuSpin: a Python package for dynamics and exact diagonalisation of quantum many body systems part I: spin chains", SciPost Phys. 2 (2017), 003.

[31] M. Goldstein, E. Sela, "Symmetry-resolved entanglement in many-body systems", Phys. Rev. Lett. 120 (2018), 200602.

[32] J. C. Xavier, F. C. Alcaraz, G. Sierra, "Equipartition of the entanglement entropy", Phys. Rev. B 98 (2018), 041106.

[33] P. Zanardi, M. Rasetti, "Noiseless quantum codes", Phys. Rev. Lett. 79 (1997), 3306–3309.

[34] E. Knill, R. Laflamme, L. Viola, "Theory of quantum error correction for general noise", Phys. Rev. Lett. 84 (2000), 2525–2528.

[35] C. Bény, A. Kempf, D. W. Kribs, "Generalization of quantum error correction via the Heisenberg picture", Phys. Rev. Lett. 98 (2007), 100502.

[36] X. Bonet-Monroig, R. Sagastizabal, M. Singh, T. E. O'Brien, "Low-cost error mitigation by symmetry verification", Phys. Rev. A **98** (2018), 062339.

[37] S. McArdle, X. Yuan, S. Benjamin, "Error-mitigated digital quantum simulation", Phys. Rev. Lett. **122** (2019), 180501.

[38] S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli, R. Biswas, "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz", Algorithms 12 (2019), 34.

[39] F. J. Dyson, "The threefold way. Algebraic structure of symmetry groups and ensembles in quantum mechanics", J. Math. Phys. 3 (1962), 1199–1215.

[40] T. Cohen, M. Welling, "Group equivariant convolutional networks", Proc. ICML 2016, PMLR 48, 2990–2999.

[41] M. Finzi, M. Welling, A. G. Wilson, "A practical method for constructing equivariant multilayer perceptrons for arbitrary matrix groups", Proc. ICML 2021, PMLR 139, 3318–3328.

[42] C. H. Martin, M. W. Mahoney, "Implicit self-regularization in deep neural networks: evidence from random matrix theory and implications for learning", J. Mach. Learn. Res. 22 (2021), 1–73.

[43] C. R. Harris et al., "Array programming with NumPy", Nature 585 (2020), 357–362.

[44] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python", Nat. Methods 17 (2020), 261–272.

[45] R. B. Lehoucq, D. C. Sorensen, C. Yang, *ARPACK Users' Guide*, SIAM, 1998.

[46] M. Fishman, S. R. White, E. M. Stoudenmire, "The ITensor software library for tensor network calculations", SciPost Phys. Codebases 4 (2022).

[47] M. M. Rams, G. Wójtowicz, A. Sinha, J. Hasik, "YASTN: Yet another symmetric tensor networks; A Python library for Abelian symmetric tensor network calculations", SciPost Phys. Codebases 52 (2025); arXiv:2405.12196.

[48] J. Hauschild, F. Pollmann, "Efficient numerical simulations with tensor networks: Tensor Network Python (TeNPy)", SciPost Phys. Lect. Notes 5 (2018).

[49] M. Fukuda, R. König, I. Nechita, "RTNI – a symbolic integrator for Haar-random tensor networks", J. Phys. A 52 (2019), 425303.

[50] N. J. Higham, *Functions of Matrices: Theory and Computation*, SIAM, 2008.

[51] L. N. Trefethen, J. A. C. Weideman, "The exponentially convergent trapezoidal rule", SIAM Rev. 56 (2014) 385–458.

[52] Ju. L. Daleckii, S. G. Krein, "Integration and differentiation of functions of Hermitian operators and applications to the theory of perturbations", Amer. Math. Soc. Transl. (2) 47 (1965) 1–30.

[53] R. Bhatia, *Matrix Analysis*, Graduate Texts in Mathematics 169, Springer, 1997.

[54] M. F. Kasim, "Derivatives of partial eigendecomposition of a real symmetric matrix for degenerate cases", arXiv:2011.04366 (2020).

[55] D. Petz, "Quasi-entropies for finite quantum systems", Rep. Math. Phys. 23 (1986), 57–65.

[56] D. Petz, "Sufficient subalgebras and the relative entropy of states of a von Neumann algebra", Commun. Math. Phys. 105 (1986), 123–131.

[57] D. Petz, "Sufficiency of channels over von Neumann algebras", Quart. J. Math. Oxford 39 (1988), no. 1, 97–108.

[58] M. Müller-Lennert, F. Dupuis, O. Szehr, S. Fehr, M. Tomamichel, "On quantum Rényi entropies: a new generalization and some properties", J. Math. Phys. 54 (2013), 122203.

[59] M. M. Wilde, A. Winter, D. Yang, "Strong converse for the classical capacity of entanglement-breaking and Hadamard channels via a sandwiched Rényi relative entropy", Commun. Math. Phys. 331 (2014), 593–622.

[60] R. L. Frank, E. H. Lieb, "Monotonicity of a relative Rényi entropy", J. Math. Phys. 54 (2013), 122201.

[61] S. Beigi, "Sandwiched Rényi divergence satisfies data processing inequality", J. Math. Phys. 54 (2013), 122202.

[62] M. Berta, O. Fawzi, M. Tomamichel, "On variational expressions for quantum relative entropies", Lett. Math. Phys. 107 (2017), 2239–2265.

[63] H. Zhang, "From Wigner–Yanase–Dyson conjecture to Carlen–Frank–Lieb conjecture", Adv. Math. 365 (2020), 107053.

[64] M. Junge, R. Renner, D. Sutter, M. M. Wilde, A. Winter, "Universal recovery maps and approximate sufficiency of quantum relative entropy", Ann. Henri Poincaré 19 (2018), 2955–2978.

[65] V. F. R. Jones, "Index for subfactors", Invent. Math. 72 (1983), 1–25.

[66] F. M. Goodman, P. de la Harpe, V. F. R. Jones, *Coxeter Graphs and Towers of Algebras*, MSRI Publications 14, Springer, 1989.

[67] M. Pimsner, S. Popa, "Entropy and index for subfactors", Ann. Sci. École Norm. Sup. (4) 19 (1986), 57–106.

[68] A. Connes, E. Størmer, "Entropy for automorphisms of II_1 von Neumann algebras", Acta Math. 134 (1975), 289–306.

[69] M. Choda, "Relative entropy for maximal abelian subalgebras of matrices and the entropy of unistochastic matrices", Internat. J. Math. 19 (2008), 767–776; arXiv:0803.0591.

[70] K. C. Bakshi, S. Guin, "Relative position between a pair of spin model subfactors", J. Aust. Math. Soc. 119 (2025), no. 1, 1–38; arXiv:2212.07197.

[71] D. Petz, A. Szántó, M. Weiner, "Complementarity and the algebraic structure of 4-level quantum systems", Infin. Dimens. Anal. Quantum Probab. Relat. Top. 12 (2009), 99–116; arXiv:0802.2391.

[72] I. Ghosh, S. Kumar, "Kadison–Kastler distance and interior angle for masas: connections with entropy and probabilistic index", arXiv:2609.38206 (2026).

[73] W. Greub, W. Rheinboldt, "On a generalization of an inequality of L. V. Kantorovich", Proc. Amer. Math. Soc. 10 (1959), 407–415.

[74] T. Furuta, J. Mićić Hot, J. Pečarić, Y. Seo, *Mond–Pečarić Method in Operator Inequalities*, Element, Zagreb, 2005.

[75] Y. Watatani, "Index for C^*-subalgebras", Mem. Amer. Math. Soc. 83 (1990), no. 424.

[76] M. Frank, E. Kirchberg, "On conditional expectations of finite index", J. Operator Theory 40 (1998), 87–111.

[77] M. Baillet, Y. Denizeau, J.-F. Havet, "Indice d'une espérance conditionnelle", Compositio Math. 66 (1988), 199–236.

[78] K. Murota, Y. Kanno, M. Kojima, S. Kojima, "A numerical algorithm for block-diagonal decomposition of matrix $*$-algebras with application to semidefinite programming", Japan J. Indust. Appl. Math. 27 (2010), 125–160.

[79] T. Maehara, K. Murota, "A numerical algorithm for block-diagonal decomposition of matrix $*$-algebras with general irreducible components", Japan J. Indust. Appl. Math. 27 (2010), 263–293.

[80] T. Maehara, K. Murota, "Algorithm for error-controlled simultaneous block-diagonalization of matrices", SIAM J. Matrix Anal. Appl. 32 (2011), 605–620.

[81] W. Fulton, J. Harris, *Representation Theory: A First Course*, Graduate Texts in Mathematics 129, Springer, 1991.

[82] G. Racah, "Theory of complex spectra. II", Phys. Rev. 62 (1942), 438–462.

[83] A. R. Edmonds, *Angular Momentum in Quantum Mechanics*, Princeton University Press, 1957.

[84] E. Lieb, D. Mattis, "Ordering energy levels of interacting spin systems", J. Math. Phys. 3 (1962), 749–751.

[85] C. K. Majumdar, D. K. Ghosh, "On next-nearest-neighbor interaction in linear chain. I", J. Math. Phys. 10 (1969), 1388–1398.

[86] C. K. Majumdar, D. K. Ghosh, "On next-nearest-neighbor interaction in linear chain. II", J. Math. Phys. 10 (1969), 1399–1402.

[87] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library", NeurIPS 2019.

[88] F. Iemini, A. Russomanno, J. Keeling, M. Schirò, M. Dalmonte, R. Fazio, "Boundary time crystals", Phys. Rev. Lett. 121 (2018), 035301.

[89] F. Carollo, I. Lesanovsky, "Exactness of mean-field equations for open Dicke models with an application to pattern retrieval dynamics", Phys. Rev. Lett. 126 (2021), 230601.

[90] G. Piccitto, M. Wauters, F. Nori, N. Shammah, "Symmetries and conserved quantities of boundary time crystals in generalized spin models", Phys. Rev. B 104 (2021), 014307.

[91] Z. Liu, Y. Li, Z. Fei, X. Wang, "Boundary time crystals beyond mean-field theory: a stroboscopic rotating-wave approximation", Phys. Rev. A 114 (2026), 012211; arXiv:2510.03028.

[92] D. Nemeth, A. Principi, A. Nazir, "Analytical solution of boundary time crystals via the superspin basis", arXiv:2507.06998 (2025).

[93] Z. Wang, R. Gao, X. Wu, B. Buča, K. Mølmer, L. You, F. Yang, "Boundary time crystals induced by local dissipation and long-range interactions", Phys. Rev. Lett. 135 (2025), 230401.

[94] K. Tucker, B. Zhu, R. J. Lewis-Swan, J. Marino, F. Jimenez, J. G. Restrepo, A. M. Rey, "Shattered time: can a dissipative time crystal survive many-body correlations?", New J. Phys. 20 (2018), 123003.

[95] L. F. dos Prazeres, L. da Silva Souza, F. Iemini, "Boundary time crystals in collective d-level systems", Phys. Rev. B 103 (2021), 184308.

[96] B. A. Chase, J. M. Geremia, "Collective processes of an ensemble of spin-1/2 particles", Phys. Rev. A 78 (2008), 052101.

[97] B. Q. Baragiola, B. A. Chase, J. Geremia, "Collective uncertainty in partially polarized and partially decohered spin-1/2 systems", Phys. Rev. A 81 (2010), 032104.

[98] N. Shammah, S. Ahmed, N. Lambert, S. De Liberato, F. Nori, "Open quantum systems with local and collective incoherent processes: Efficient numerical simulations using permutational invariance", Phys. Rev. A 98 (2018), 063815.

[99] M. Gegg, M. Richter, "PsiQuaSP – a library for efficient computation of symmetric open quantum systems", Sci. Rep. 7 (2017), 16304.

[100] P. Kirton, J. Keeling, "Suppressing and restoring the Dicke superradiance transition by dephasing and decay", Phys. Rev. Lett. 118 (2017), 123602.

[101] R. H. Dicke, "Coherence in spontaneous radiation processes", Phys. Rev. 93 (1954), 99–110.

[102] M. Gross, S. Haroche, "Superradiance: an essay on the theory of collective spontaneous emission", Phys. Rep. 93 (1982), 301–396.

[103] A. I. Stolyarov, O. E. Tikhonov, A. N. Sherstnev, "Characterization of normal traces on von Neumann algebras by inequalities for the modulus", Math. Notes 72 (2002), no. 3–4, 411–416.

[104] A. M. Bikchentaev, O. E. Tikhonov, "Characterization of the trace by Young's inequality", J. Inequal. Pure Appl. Math. 6 (2005), no. 2, Art. 49.

[105] A. M. Bikchentaev, O. E. Tikhonov, "Characterization of the trace by monotonicity inequalities", Linear Algebra Appl. 422 (2007), no. 1, 274–278.

[106] A. M. Bikchentaev, "The Peierls–Bogoliubov inequality in C*-algebras and characterization of tracial functionals", Lobachevskii J. Math. 32 (2011), no. 3, 175–179.

[107] A. M. Bikchentaev, "Commutation of projections and trace characterization on von Neumann algebras. II", Math. Notes 89 (2011), no. 3–4, 461–471.

[108] A. M. Bikchentaev, "Commutativity of projections and characterization of traces on von Neumann algebras", Sib. Math. J. 51 (2010), no. 6, 971–977.

[109] K. Cho, T. Sano, "Young's inequality and trace", Linear Algebra Appl. 431 (2009), no. 8, 1218–1222.

[110] A. M. Bikchentaev, S. A. Abed, "Projections and traces on von Neumann algebras", Lobachevskii J. Math. 40 (2019), no. 9, 1260–1267.

[111] A. M. Bikchentaev, "Metrics on projections of the von Neumann algebra associated with tracial functionals", Sib. Math. J. 60 (2019), 952–956.

[112] A. M. Bikchentaev, "Inequalities for determinants and characterization of the trace", Sib. Math. J. 61 (2020), no. 2, 248–254.

[113] A. M. Bikchentaev, M. S. Moslehian, "Trace inequalities and characterizations of tracial functionals in operator algebras", Positivity 30 (2026), Art. 24; doi:10.1007/s11117-026-01180-0.

[114] A. M. Bikchentaev, M. S. Moslehian, "Characterizations of tracial functionals on matrix and operator algebras", Bull. Belg. Math. Soc. Simon Stevin 32 (2025), no. 3, 343–361.

[115] A. M. Bikchentaev, T. H. Dinh, A. V. Le, M. S. Moslehian, "Spectral geometric mean and trace characterizations", Positivity 30 (2026), Art. 46.

[116] N. Laflorencie, S. Rachel, "Spin-resolved entanglement spectroscopy of critical spin chains and Luttinger liquids", J. Stat. Mech. (2014), P11013.

[117] R. Bonsignori, P. Ruggiero, P. Calabrese, "Symmetry resolved entanglement in free fermionic systems", J. Phys. A 52 (2019), 475302.

[118] S. Fraenkel, M. Goldstein, "Symmetry resolved entanglement: exact results in 1D and beyond", J. Stat. Mech. (2020), 033106.

[119] B. Estienne, Y. Ikhlef, A. Morin-Duchesne, "Finite-size corrections in critical symmetry-resolved entanglement", SciPost Phys. 10 (2021), 054.

[120] H. F. Song, S. Rachel, C. Flindt, I. Klich, N. Laflorencie, K. Le Hur, "Bipartite fluctuations as a probe of many-body entanglement", Phys. Rev. B 85 (2012), 035409.

[121] A. Lukin, M. Rispoli, R. Schittko, M. E. Tai, A. M. Kaufman, S. Choi, V. Khemani, J. Léonard, M. Greiner, "Probing entanglement in a many-body-localized system", Science 364 (2019), 256–260.

[122] M. Kiefer-Emmanouilidis, R. Unanyan, M. Fleischhauer, J. Sirker, "Evidence for unbounded growth of the number entropy in many-body localized phases", Phys. Rev. Lett. 124 (2020), 243601.

[123] K. Chen, W. Jia, X. Li, R. Meyer, J. Zhao, "Detecting symmetry-resolved entanglement: a quantum Monte Carlo approach", arXiv:2604.02307 (2026).

[124] O. E. Tikhonov, "Subadditivity inequalities in von Neumann algebras and characterization of tracial functionals", Positivity 9 (2005), no. 2, 259–264.

[125] Dinh Trung Hoa, O. E. Tikhonov, "Weighted trace inequalities of monotonicity", Lobachevskii J. Math. 26 (2007), 63–67.

[126] Dinh Trung Hoa, O. E. Tikhonov, "Weighted monotonicity inequalities for traces on operator algebras", Math. Notes 88 (2010), no. 1–2, 177–182; Mat. Zametki 88 (2010), no. 2, 193–200.

[127] T. Sano, T. Yatsu, "Characterizations of tracial property via inequalities", J. Inequal. Pure Appl. Math. 7 (2006), no. 1, Art. 36.

[128] Dinh Trung Hoa, H. Osaka, Ho Minh Toan, "On generalized Powers–Størmer's inequality", Linear Algebra Appl. 438 (2013), no. 1, 242–249.

[129] M. S. Moslehian, A. M. Bikchentaev, "Characterizations of the canonical trace on full matrix algebras", J. Math. Anal. Appl. 552 (2025), no. 2, 129764.

[130] P. R. Halmos, "Two subspaces", Trans. Amer. Math. Soc. 144 (1969), 381–389.

[131] A. M. Bikchentaev, "Commutativity of projectors and trace characterization on von Neumann algebras. I", Russian Math. (Iz. VUZ) 53 (2009), no. 12, 68–71.